

Monte Carlo — especificación completa de datos crudos

Propósito de este documento. Este documento describe, con precisión de fórmula, **todo** número que el módulo Monte Carlo de AlgoProject produce para una estrategia: qué es, de qué otro número sale, con qué operación exacta, y bajo qué modelo. **No contiene conclusiones, ni juicios de "bueno" o "malo", ni recomienda ningún umbral.** El sistema de veredicto que existe hoy en el código (`gates.py`, `scoring.py`) se documenta aparte, en el Apéndice A, etiquetado explícitamente como "el sistema actual" — es información sobre lo que ya se probó, no una respuesta a copiar.

Generado a partir del código fuente el 2026-09-13, contra la configuración de `strategies/monteCarlo/config.yaml` vigente en esa fecha (ver Apéndice C para el archivo completo). Si el código o la configuración cambian después de esta fecha, algunas fórmulas o valores por defecto de aquí pueden haber quedado desactualizados frente al código real.

Este documento también tiene ejemplos visuales reales, no inventados: cada figura que aparece aquí se generó corriendo el código de verdad sobre Strategy 17.18.29 (databank Results, XAUUSD, export 2026-09-03, `n_sims=100000`). El Apéndice E trae, además, el informe HTML completo de esa misma estrategia, renderizado a PDF página por página, tal y como lo vería quien abra el archivo `.html` en un navegador — con lo que un agente que sólo puede leer texto ve exactamente lo mismo que vería un humano abriendo el informe.

0. Dónde encaja este análisis

Antes de llegar aquí, una estrategia ya pasó — o no — por otras dos comprobaciones independientes. Se describen aquí solo en qué consisten, sin sus números: ese detalle no es parte de este documento.

- **Decaimiento IS/OOS** (`strategies/crossmarket...` realmente `tasks/` y el módulo de decaimiento): compara el rendimiento de la estrategia en el periodo con el que se construyó (in-sample) contra un periodo que no vio (out-of-sample). Mide si el Sharpe OOS se distingue de cero con su propio error estándar, si la estrategia ganó en la mayoría de los años del OOS por separado, y qué fracción del beneficio total sale de un único trimestre. Termina en una sentencia por estrategia: MANTENER, DUDOSA O DESCARTAR.
- **Retest cross-market**: coge el mismo patrón de entradas de la estrategia (mismo número de operaciones, misma duración, mismos huecos entre ellas, mismo día de la semana y hora) y lo coloca en momentos aleatorios de mercados que la estrategia nunca vio durante su optimización. Si la versión real queda entre las mejores de miles de versiones aleatorias, sus entradas llevan información sobre el momento de entrar; si no, lo que había era la deriva del propio mercado.

El módulo Monte Carlo que documenta el resto de este PDF **no repite ninguna de esas dos preguntas**. Da por hecho que la estrategia ya tiene un edge y pregunta de qué depende ese edge: del orden en que llegaron las operaciones, de cuáles llegaron, de si la ejecución fue tan buena como en el backtest, y del régimen de mercado en el que vivió.

1. El contrato de entrada: qué es una "estrategia" para este módulo

Todo lo que sigue opera sobre un **stream**: una lista de operaciones de una estrategia, ordenada en el tiempo, reducida a un conjunto fijo de arrays alineados (`strategies/monteCarlo/stream.py`). Un stream no distingue entre una estrategia y una cartera: `stream.portfolio()` concatena varias estrategias por tiempo y todo lo demás corre igual sobre el resultado.

Por cada operación, el stream guarda:

campo	de dónde sale	fórmula
pnl	columna Profit/Loss del CSV que exporta SQX	tal cual, en USD, neto
r	pnl dividido por el riesgo configurado	$r = \text{pnl} / \text{risk_per_trade}$
cost	comisión + swap que SQX cobró de verdad, recuperada , no asumida	$\text{cost} = \text{gross} - \text{net}$ de esa operación (core.trades.cost), en USD
spread	lo que costaría un spread ancho concreto en esa operación	$\text{spread_cost} = \text{spread_puntos} \times \text{tick_size} \times \text{point_value} \times \text{Size}$
mae	columna MAE (\$) del CSV (adverse excursion máxima de la operación)	tal cual
size	columna Size del CSV (lotes)	tal cual
open, close	columnas Open time / Close time	tal cual
sample	columna Sample type que SQX escribe (IST = in-sample, OOS1 = fuera de muestra)	los tres primeros caracteres deciden IS/OOS

Por qué el coste se recupera y no se asume. El export de SQX no trae una columna de coste. En vez de tomar el spread/commission de la ficha del activo (assets/<SYMBOL>.yaml) como el coste real, se resta neto de bruto: eso es exactamente lo que SQX cobró en cada operación durante el backtest, y es sobre esas cifras sobre las que tiene sentido preguntar "¿y si el broker fuera peor?" (Familia C). La ficha del activo sólo entra para modelar spread y para la comprobación cruzada del apartado 1.1.

1.1 Comprobación cruzada de coste (costs.crosscheck)

```

modelado = 2 × comisión_ficha × Size (mediana sobre todas las operaciones)
recuperado = gross - net (mediana sobre todas las operaciones)
ratio = modelado / recuperado
diverge = |ratio - 1| > 0.25

```

Si diverge es cierto, la ficha del activo en assets/ describe un instrumento distinto del que se operó de verdad. Esto no invalida el análisis entero: sólo la Familia C, que sí usa la ficha para modelar spread y comisión peores.

2. El motor de simulación

Cada "sub-prueba" consiste en simular **N rutas** (n_sims, 100.000 por defecto) de una operación sobre las mismas operaciones, bajo un modelo concreto, y reducir cada ruta a ocho estadísticos (sección 2.3). El motor (engine.py) reparte las N rutas en lotes (chunk, 2.000 por defecto) para acotar la memoria, y usa un pool de procesos (uno por núcleo) para las pruebas grandes.

2.1 Los cinco modelos de reordenación/remuestreo (draws.py) — Familias A y B

Cada modelo recibe (número de simulaciones n, número de operaciones size, generador aleatorio, longitud de bloque block) y devuelve una matriz de n filas: en cada fila, los **índices** de las operaciones originales que forman esa ruta simulada, en el orden que le tocó.

- iid_shuffle — permutación uniforme de las size operaciones. Cada operación aparece exactamente una vez por ruta; sólo cambia el orden. Preserva el beneficio neto exactamente (sweeps.invariant()) mide esto: la desviación estándar del beneficio neto entre rutas es cero hasta el error de coma flotante).
- block_shuffle — se cortan las operaciones en bloques consecutivos de block operaciones (el último bloque se rellena y se recorta), se permutan los bloques enteros, y dentro de cada bloque el orden interno no cambia.

También preserva el beneficio neto exactamente: cada operación sigue apareciendo una sola vez.

- **stationary** (bootstrap estacionario de Politis-Romano) — para cada ruta y cada posición, con probabilidad $1/\text{block}$ se sortea una posición de arranque nueva al azar; si no, la ruta continúa en la posición siguiente a la anterior, dando la vuelta al final de la serie $((\text{pos} + 1) \% \text{size})$. El resultado: la longitud media de cada tramo contiguo es `block`, pero es geométrica, no fija, y **no es una permutación** — una misma operación puede aparecer varias veces en una ruta y otra ninguna. Por eso **no** preserva el beneficio neto exactamente; su distribución de beneficio neto varía de ruta en ruta, a diferencia de los dos modelos anteriores. Es el modelo de cabecera de la Familia A (constante `HEADLINE` en `sweeps.py`): su distribución de drawdown es la que leen los apartados que hablan de "el drawdown reordenado".
- **iid_bootstrap** — cada posición de cada ruta se sortea de forma independiente y uniforme, con reemplazo, entre las `size` operaciones. Es el modelo base (BASELINE) de la Familia B: cambia qué operaciones aparecen y cuántas veces, sin ninguna estructura de bloque. `block` se ignora.
- **block_bootstrap** — se sortean posiciones de inicio de bloque al azar, con reemplazo, y cada bloque contiguo (dando la vuelta al final si hace falta) se copia entero. Preserva rachas de operaciones consecutivas mientras cambia la composición — la versión de la Familia B que respeta la correlación entre operaciones vecinas.

`block` sólo tiene efecto en `block_shuffle`, `stationary` y `block_bootstrap`. Los tamaños de bloque que se prueban salen de `config.block_sizes()`: `n_block_sizes` valores (6 por defecto) repartidos uniformemente entre `block_min` (5 por defecto) y `floor(N_operaciones / min_blocks)` (`min_blocks` = 20 por defecto); si ese máximo es menor que `block_min`, el barrido de bloques se omite. El bloque del modelo `stationary` es distinto: `config.stationary_block()` devuelve `round(N_operaciones^(1/3))` (redondeado, mínimo 2), la regla práctica de Politis-Romano.

2.2 Los cuatro estrés de ejecución (`stress.py`) — Familia C

A diferencia de los modelos anteriores, cada función de estrés devuelve directamente una matriz de `n rutas × size` operaciones de **P&L perturbado** (no índices):

- **skip** — cada operación de cada ruta se pierde al azar con probabilidad `p_skip` (0.05 por defecto); una operación perdida cuenta como 0, no se elimina (la ruta conserva su longitud). $\text{pnl_ruta} = \text{pnl} \times (\text{aleatorio} \geq \text{p_skip})$.
- **cost_shock** — a cada operación se le aplica un multiplicador de coste sorteado uniformemente en `cost_shock_range` (1.0–2.0 por defecto), y sólo se resta el **incremento** sobre el coste ya recuperado (el P&L ya es neto, restar el coste entero lo contaría dos veces): $\text{pnl_ruta} = \text{pnl} - \text{cost} \times (\text{multiplicador} - 1)$.
- **fill_degrade** — con probabilidad `fill_frac` (0.15) cada operación "sufre" una ejecución peor: se le quita una fracción, sorteada uniformemente entre 0 y `fill_depth` (0.25), de la distancia entre su P&L y su propio MAE (lo peor que llegó a estar esa operación en contra): $\text{pnl_ruta} = \text{pnl} - \text{profundidad} \times (\text{pnl} - \text{mae})$, sólo en las operaciones "tocadas". El límite es el propio MAE de la operación: nunca se inventa una pérdida mayor que la que el mercado realmente llegó a poner en contra.
- **spread_widen** — a cada operación se le aplica un multiplicador de spread sorteado uniformemente en `spread_scale` (1.0–2.0), aplicado a **todas** las operaciones, no a una muestra: $\text{pnl_ruta} = \text{pnl} - \text{spread_cost} \times \text{multiplicador}$.

Cada una de las cuatro corre siempre, sobre las 100.000 rutas por defecto, aunque otra ya haya fallado.

2.3 Los ocho estadísticos que se calculan de cada ruta simulada (`metrics.paths`)

Dado el P&L por operación de una ruta (o de miles a la vez) y la cuenta inicial `equity0`:

```
equity = equity0 + cumsum(pnl) # curva de equity aditiva
peak = máximo_acumulado_de_equity_hasta_cada_punto
drop = peak - equity
net = suma(pnl)
return_pct = net / equity0
dd = máximo(drop) # drawdown máximo, en $
```

```
dd_pct = máximo(drop / peak) # drawdown máximo, como fracción de la cuenta
ret_dd = net / dd (NaN si dd = 0)
sharpe = media(pnl) / desviación_estándar(pnl, ddof=1) # por operación, sin anualizar
pf = suma(ganancias) / suma(|pérdidas|) (NaN si no hay pérdidas)
losing_run = racha más larga de operaciones consecutivas con pnl < 0
```

El **backtest real** se reduce a estos mismos ocho números pasando su única fila de P&L por la misma función (`metrics.observed`) — es el número contra el que se compara cada distribución simulada.

2.4 De miles de rutas a lo que se guarda (`metrics.summarise`, `metrics.shape`)

Para cada estadístico de cada sub-prueba se guardan dos resúmenes distintos, nunca las rutas crudas:

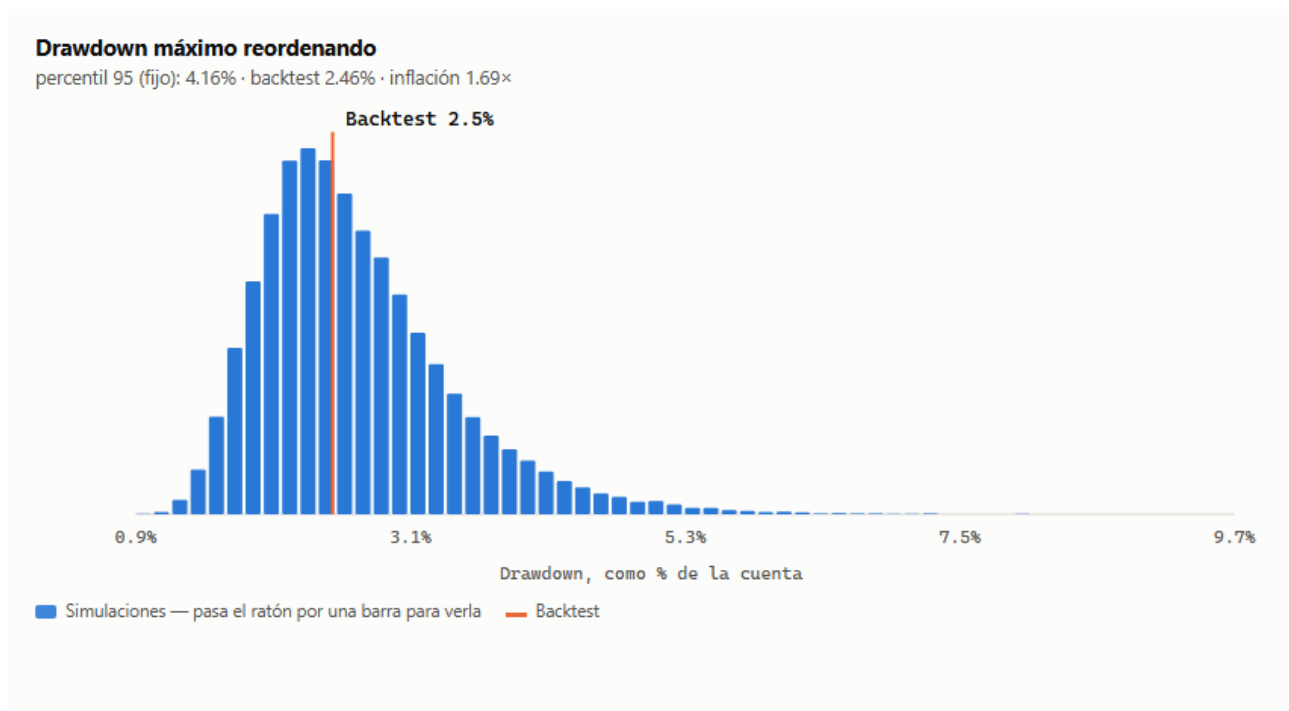
`summarise()` (usado en las tablas):

```
p = {percentil: valor, para cada percentil de global.percentile_set} # [1,5,25,50,75,95,99] por defecto
mean, median, std = media, mediana y desviación estándar (ddof=1) de la distribución simulada
kurtosis = curtosis exceso: media(((x - media)/std)^4) - 3 (0 si std = 0)
observed = el valor del backtest real para ese estadístico
rank = fracción de rutas simuladas con valor ≤ el del backtest
n = simulaciones utilizables (después de quitar NaN/Inf)
```

`shape()` (usado para dibujar el histograma):

```
counts, lo, hi = un histograma de 60 bins que cubre el rango de la distribución,
ensanchado si hace falta para incluir siempre el valor del backtest
observed = el valor del backtest real
median = mediana de la distribución simulada
p_report = percentil global.report_percentile (95 por defecto) de la distribución simulada
```

Así se ve un **histograma real** (el de la sección 3, drawdown reordenado de Strategy 17.18.29):



Cada barra es traducible: pasar el ratón por encima (en el HTML real, no en este PDF) enseña el rango de esa barra, cuántas simulaciones cayeron ahí y el percentil acumulado hasta ese punto. La línea vertical es siempre el backtest real, nunca una simulación.

3. Familia A — suerte de orden (run.py: _family_a, familyd no interviene aquí)

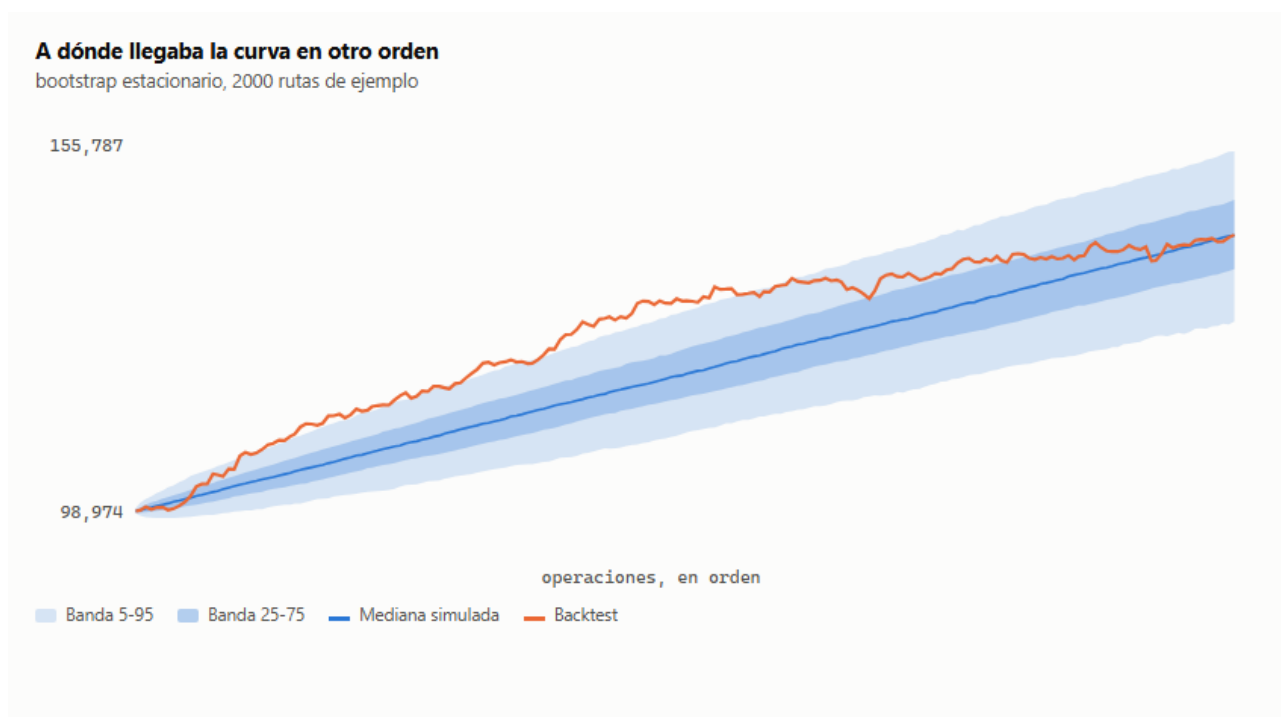
Corre los modelos iid_shuffle, block_shuffle (todos los tamaños de bloque) y stationary (modelo de cabecera) sobre las operaciones completas, a n_sims rutas cada uno.

```
dd_pct_95 = percentil 95 de dd_pct bajo el modelo de cabecera (stationary) – SIEMPRE percentil 95, fijo, no configurable
dd_pct_99 = percentil 99 de dd_pct bajo el modelo de cabecera – SIEMPRE percentil 99, fijo
inflation = dd_pct_95 / dd_pct_observado_en_el_backtest
```

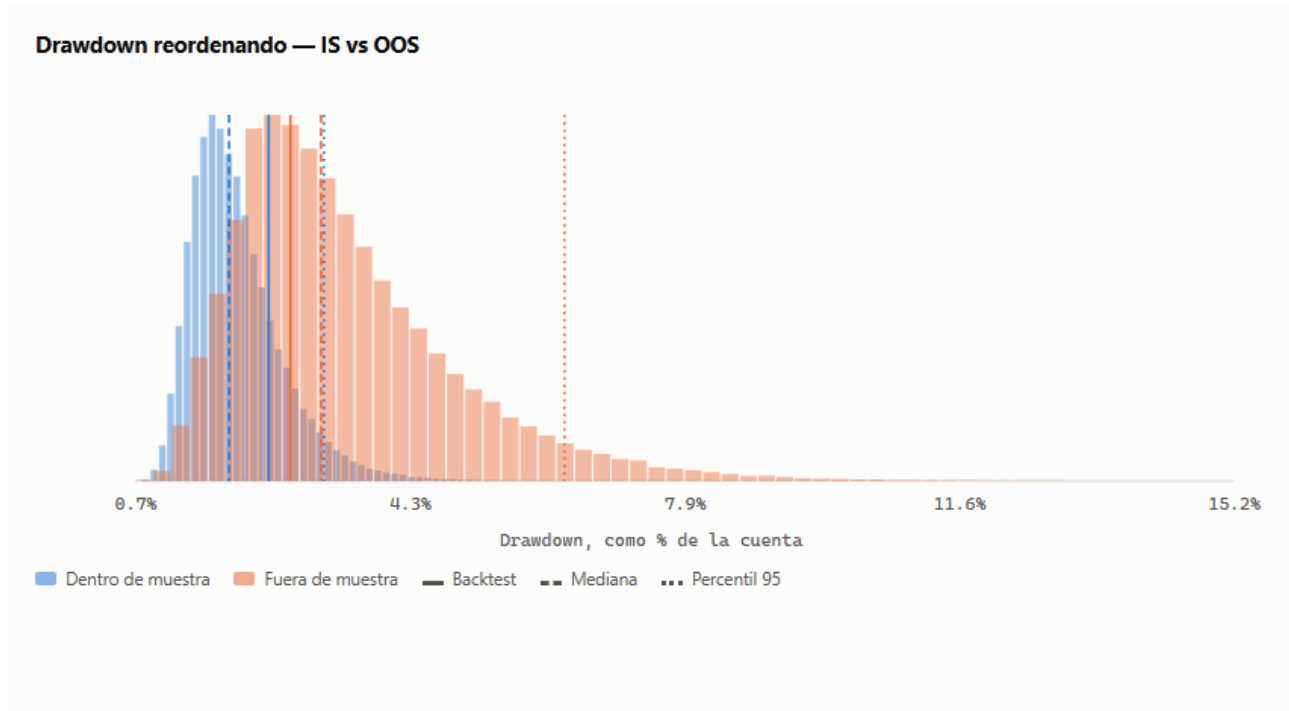
Además, por cada modelo/tamaño de bloque: la tabla completa de summarise() de los ocho estadísticos, y el histograma (shape()) de cada uno.

invariant mide, para iid_shuffle y cada tamaño de block_shuffle, la desviación estándar del beneficio neto entre rutas — debe ser cero salvo error de coma flotante, por construcción (sección 2.1).

La curva de equity bajo el modelo de cabecera, con la banda 5-95 y 25-75 de dónde podría haber llegado la misma secuencia de operaciones en otro orden, y el backtest real encima:



El histograma solapado IS/OOS de esta misma familia (sección 8 explica el mecanismo general):



En este caso concreto la distribución OOS (naranja) está claramente desplazada a la derecha de la IS (azul) — el drawdown reordenado que le tocaría a la parte fuera de muestra es sistemáticamente peor que el de la parte con la que se construyó la estrategia. Es exactamente el tipo de patrón que esta figura existe para hacer visible de un vistazo.

4. Familia B — suerte de composición (run.py: _family_b)

Corre iid_bootstrap (modelo base) y block_bootstrap (todos los tamaños de bloque) sobre las operaciones completas:

net_5 = mínimo, entre todos los modelos de esta familia, del percentil 5 de net – percentil 5, fijo
 pf_5 = mínimo, entre todos los modelos de esta familia, del percentil 5 de pf – percentil 5, fijo

Dependencia de la mejor operación (no es una simulación, es aritmética directa sobre el backtest real):

```
net_sin_mejor = suma(pnl) – máximo(pnl)
share = 1 – net_sin_mejor / suma(pnl) (NaN si suma(pnl) = 0)
```

Dentro y fuera de muestra (stream.samples() separa las operaciones por su etiqueta IST/ OOS1; si un lado tiene menos de confidence.MEAN_PROVISIONAL = 10 operaciones, se omite): para cada lado por separado, se corre iid_bootstrap sobre **sólo esas operaciones**, a n_sims rutas, y se guarda:

```
n = operaciones de ese lado
sharpe = mediana de sharpe simulado
net = mediana de net simulado
pf_5 = percentil 5 de pf simulado

oos_ratio = sharpe_mediano_OOS / sharpe_mediano_IS (NaN si falta un lado o si IS es 0)
```

5. Familia C — suerte de ejecución (run.py: _family_c)

Cada uno de los 4 estrés de la sección 2.2 corre por separado sobre las operaciones completas, a n_sims rutas:

```
median_net = mediana de net simulado
net_5 = percentil 5 de net simulado – fijo
pf_5 = percentil 5 de pf simulado – fijo
keep = median_net / net_observado_en_el_backtest
```

Más la tabla `summarise()` y el histograma `shape()` de los ocho estadísticos, por cada uno de los 4 estrés.

6. Familia D — suerte de régimen (`familyd.py`)

Todo lo de esta familia usa `stationary` (no `iid_bootstrap`) a `family_d.window_sims` rutas (20.000 por defecto), no a `global.n_sims`, con el bloque de cada ventana calculado sobre su propio tamaño (`config.stationary_block(n_operaciones_de_la_ventana)`, la misma regla que usa la Familia A). **Cambiado en esta sesión** — antes usaba `iid_bootstrap`, que trata cada operación de la ventana como independiente. Una ventana son dos años, lo bastante corto como para que las operaciones de dentro estén correlacionadas (una racha, un régimen que dura semanas); asumir independencia deshace esa correlación al remuestrear y da un percentil 5 más optimista del que la estrategia puede sufrir de verdad. Esto **sí** puede mover el `gate windows` y el sub-score D (Apéndice A), a diferencia del cambio de la sección 6.3, que es puramente narrativo.

6.1 Ventanas de calendario (`windows.rolling`)

Una ventana cubre `family_d.window_months` meses (24 por defecto) de calendario — no de operaciones —, empezando en el primer mes completo de operativa. Las **ventanas móviles** ("overlapping") avanzan `family_d.window_step_months` meses (6 por defecto) entre una y la siguiente; los **bloques no solapados** ("nonoverlapping") avanzan `window_months` meses completos, así que ninguna operación pertenece a dos bloques. Una ventana que se saldría del final de los datos se descarta entera, nunca se acorta.

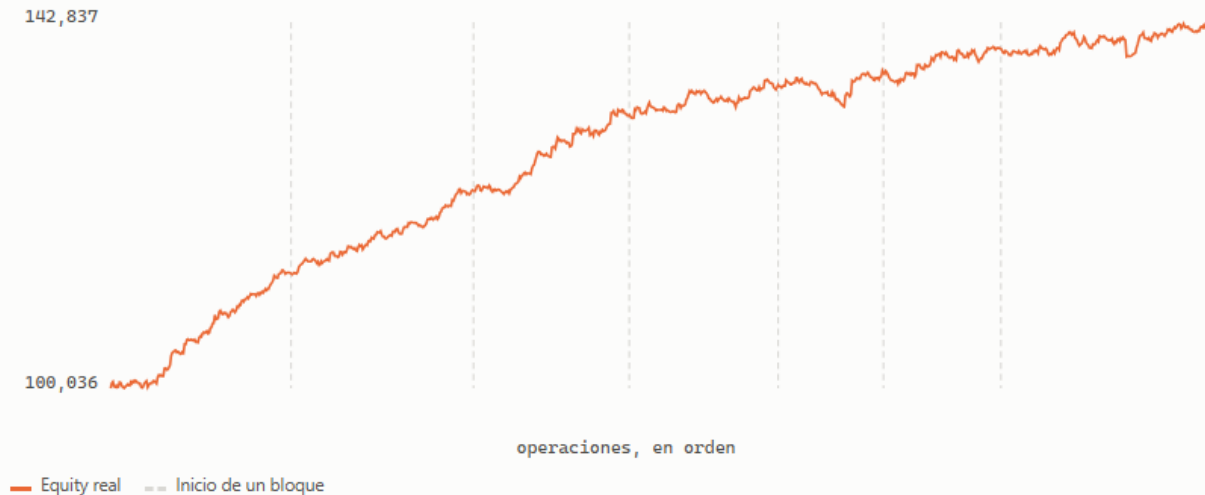
Por cada ventana (con al menos `confidence.MEAN_PROVISIONAL = 10` operaciones dentro; si no, sale con todos los números en NaN):

```
n = operaciones dentro de la ventana
median_net = mediana de net bajo stationary sobre sólo esas operaciones
net_5 = percentil 5 de net – fijo
pf_5 = percentil 5 de pf – fijo
shape = histograma de net (solo para los bloques no solapados, no para las ventanas móviles)
```

Curva de equity con marcas: la curva de equity real completa (`equity0 + cumsum(pnl)`, en el orden real de las operaciones) más la posición (índice de operación, no fecha) donde empieza cada bloque no solapado.

Curva de equity con las ventanas marcadas

cada línea discontinua abre un bloque de la tabla de arriba



6.2 Régimen de volatilidad (regime.py)

La volatilidad se calcula sobre velas **diarias** (regime.daily() colapsa las barras intradía a un OHLC por día), sea cual sea el timeframe en el que opera la estrategia.

- **ATR** (vol_model: atr, por defecto): rango verdadero de Wilder, $TR = \text{máximo}(\text{high} - \text{low}, |\text{high} - \text{cierre_anterior}|, |\text{low} - \text{cierre_anterior}|)$, suavizado con una media móvil exponencial de parámetro $1/\text{atr_period}$ (20 días por defecto).
- **GARCH** (vol_model: garch): volatilidad condicional de un modelo GARCH(p,q) (orden garch_order, por defecto (1,1)) sobre los retornos logarítmicos diarios $\times 100$, con distribución de residuos garch_dist (t-Student por defecto). Si no converge, cae a ATR y lo dice en note.

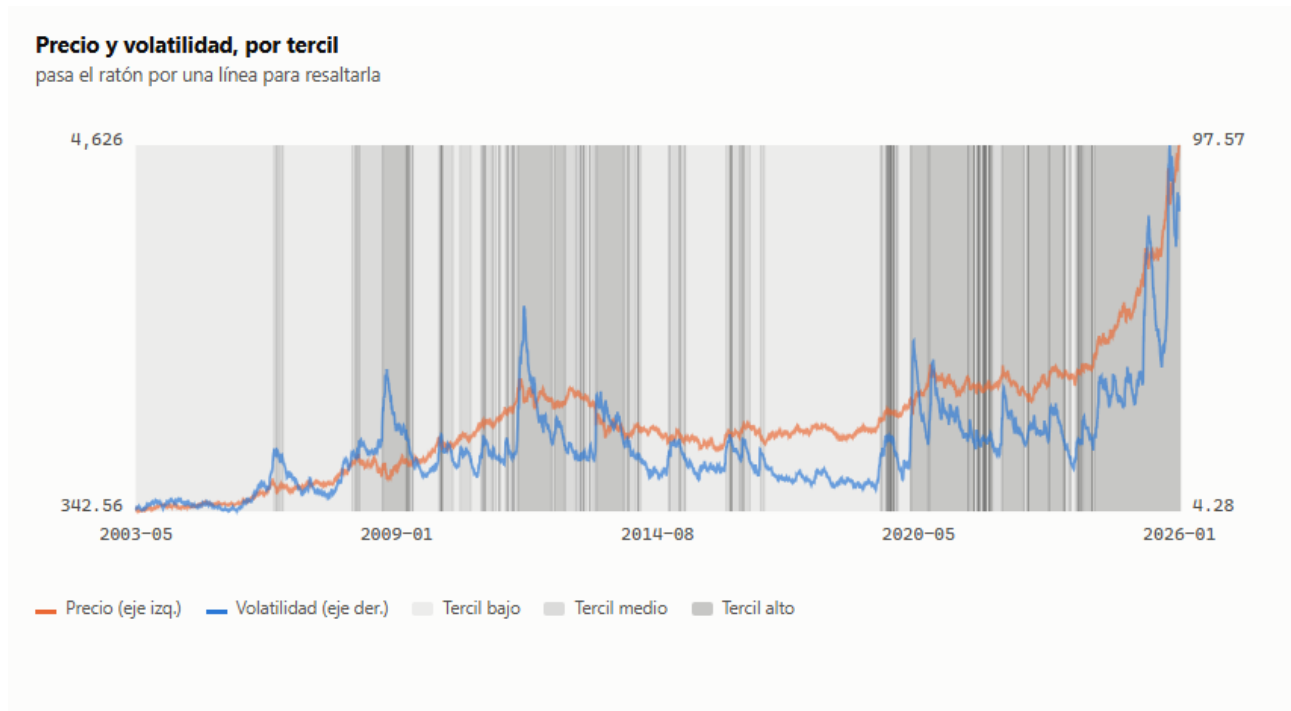
Los **terciles** (regime.tag) se calculan sobre los días en los que la estrategia tuvo alguna operación abierta: edges = percentiles [33.3, 66.7] de la volatilidad de esos días. Una operación que abre un día sin dato de volatilidad en el archivo de barras **hace fallar el análisis entero** con un error explícito — no se imputa ningún valor.

Por cada tercil (low, mid, high):

```
n = operaciones abiertas en un día de ese tercil
net = suma(pnl) real de esas operaciones (no simulado)
median_net = mediana de net bajo stationary sobre sólo esas operaciones
net_5 = percentil 5 de net — fijo
pf_5 = percentil 5 de pf — fijo
shape = histograma de net

concentration = máximo(net de un tercil) / suma(pnl total)
coverage = fracción de operaciones cuyo día sí tenía dato de volatilidad
```

Serie de precio y volatilidad: para cada día en el que el modelo de volatilidad tiene valor (no NaN — los primeros atr_period días no lo tienen), se guarda su fecha, el cierre diario, el valor de volatilidad de ese día, y a qué tercil pertenece (0=bajo, 1=medio, 2=alto, según los mismos edges de arriba).



6.3 El peor camino posible, a varias severidades (`stitch.worst_path`)

Cambiado en esta sesión. Para cada bloque no solapado con más de una operación: se sortean `window_sims` remuestreos con reemplazo de las operaciones de **ese bloque solo**. Antes se ordenaban por beneficio neto total y se quedaba uno solo, el situado en la posición $0.05 \times (\text{window_sims} - 1)$ (percentil-5 discreto, no interpolado) — un número fijo, sin argumento de principio para por qué 5% y no 1% o 25%. Ahora se ordenan **una sola vez** y se leen varias posiciones de ese mismo orden, una por cada valor de `family_d.stitch_quantiles` (por defecto `[0.01, 0.05, 0.10, 0.25]`) — mismo remuestreo compartido entre las cuatro severidades, así que la única diferencia entre columnas es qué tan adversa es la posición elegida, no una tirada de dados distinta. Los remuestreos elegidos de cada bloque se **concatenan** en un único camino continuo por severidad, y cada camino se reduce a los ocho estadísticos de la sección 2.3 (con `equity0` real). No es un resultado de Monte Carlo global: es "¿cómo sería si cada periodo, por separado, hubiera tenido uno de sus peores remuestreos?" — un camino de estrés, nunca promedio de nada, y nunca entra en ningún gate ni sub-score (Apéndice A) — es sólo narrativa.

Ejemplo real, la misma tabla que aparece en el informe:

percentil por bloque	drawdown	beneficio neto	bloques cosidos
1%	17.2%	-6 918 \$	7
5%	10.7%	6 753 \$	7
10%	5.3%	12 963 \$	7
25%	3.7%	26 159 \$	7

Leído crudo: en esta estrategia el drawdown del camino cosido pasa de 17.2% a 3.7% según qué tan adversa se elija la severidad — la referencia de estrés **no es estable** frente a esa elección, lo cual es en sí mismo un hecho sobre esta estrategia, no un defecto del método.

6.4 Calendario (`windows.calendar`)

Beneficio real (no simulado) sumado por mes del año (1–12) y por día de la semana (0=lunes–4=viernes). Sólo diagnóstico: con 12 meses, el mejor de un reparto puramente aleatorio ya destaca.



File not found

It may have been moved, edited, or deleted.

ERR_FILE_NOT_FOUND

7. Familia E — significación (significance.py)

Ninguno de estos cuatro números es una simulación: son una fórmula analítica aplicada una única vez sobre las operaciones reales del backtest, en su propio orden:

```
sr = media(pnl) / desviación_estándar(pnl, ddof=1) # Sharpe por operación
skew = asimetría de la muestra de pnl (scipy.stats.skew)
kurt = curtosis de la muestra de pnl, NO exceso (scipy.stats.kurtosis(fisher=False); una normal da 3)
n = número de operaciones

var = 1 - skew*sr + (kurt-1)/4 * sr2
z = (sr - benchmark) * √(n-1) / √var # benchmark = family_e.psr_benchmark, 0 por defecto
psr = Φ(z) # Φ = función de distribución normal estándar acumulada
```

La PSR (Probabilistic Sharpe Ratio, de Bailey & López de Prado) es la probabilidad de que el Sharpe verdadero de la estrategia supere benchmark, corrigiendo por cuántas operaciones hay y por la forma exacta de la distribución (asimetría y colas). Es un estimador puntual, no una distribución: no se remuestrea, porque hacerlo contaría la misma incertidumbre muestral dos veces.

La única cifra simulada de esta familia es la comprobación cruzada contra el remuestreo de la Familia B:

```
bootstrap = fracción de rutas de iid_bootstrap (el modelo base de la Familia B) cuyo sharpe simulado > 0
gap = |psr - bootstrap|
```

Un gap grande dice que las operaciones son lo bastante asimétricas o con colas lo bastante gordas como para que la aproximación normal de la PSR y el remuestreo empírico no coincidan; no es un error de ninguno de los dos métodos.

No hay Deflated Sharpe Ratio en ningún sitio de este módulo: haría falta saber cuántas estrategias se probaron durante la generación, dato que no existe en esta fase.

8. La comprobación IS/OOS de cada familia (`degrade.py`)

Además de sus propias pruebas, las familias A, B, C y D tienen cada una un histograma adicional que compara la misma prueba corrida **sólo sobre las operaciones IS** contra **sólo sobre las OOS** (mismo umbral de mínimo 10 operaciones por lado que en la sección 4). El modelo de cada familia:

familia	modelo	estadístico
A	stationary, bloque = <code>blocks.block_min</code>	dd_pct
B	iid_bootstrap	net
C	cada uno de los 4 estrés, por separado	net
D	iid_bootstrap (mismo modelo y números que B; ver nota abajo)	net

Cada combinación se corre a `n_sims` rutas completas, una vez por lado (IS, OOS). El resultado de D es **numéricamente idéntico** al de B — es el mismo remuestreo sobre las mismas operaciones —, repetido en la página de D porque esa familia es la que organiza la lectura por tiempo. Para cada lado se guarda el mismo `shape()` de la sección 2.4, más el número de operaciones de ese lado. Esto es **independiente** del cambio de la sección 6 a stationary: la comprobación IS/OOS de D sigue usando iid_bootstrap a propósito, precisamente porque busca ser el mismo número que B, no una pregunta nueva sobre ventanas.

9. Fiabilidad de cada número (`confidence.py`)

Cada estadístico de este documento lleva asociado, en el código, un nivel de confianza según el tamaño de muestra del que sale — no un juicio sobre el número en sí, sino sobre si vale la pena leerlo:

```
percentil(n, q):
    cola = n * min(q, 100-q) / 100
    "reliable" si cola ≥ 10
    "provisional" si 4 ≤ cola < 10
    "unreliable" si cola < 4

media_o_mediana(n):
    "reliable" si n ≥ 30
    "provisional" si 10 ≤ n < 30
    "unreliable" si n < 10

bloques(n, block, min_blocks):
    "reliable" si n // block ≥ min_blocks
    "unreliable" en caso contrario
```

El peor nivel entre todos los números que alimentan una decisión es el nivel de esa decisión (`confidence.worst`) — nunca un promedio.

10. Estabilidad del propio Monte Carlo (`stability.py`)

Como no hay semilla fija (cada ejecución usa entropía nueva a propósito), se mide cuánto se mueven los números que más importan si se recalculan de forma independiente: `stability.n_stability_runs` veces (8 por defecto) se recalculan, a las mismas `n_sims` rutas completas cada vez, estos cuatro números concretos: dd_pct al percentil 95 y al 99 bajo stationary, net al percentil 5 y pf al percentil 5 bajo iid_bootstrap, más los mismos cuatro bajo cada uno

de los 4 estrés de la Familia C. Por cada uno se guarda la media entre las 8 repeticiones y la dispersión relativa (máximo – mínimo) / |media|. Si la dispersión del peor de todos supera `stability.stability_tol` (10% por defecto), se marca como inestable.

Apéndice A — El sistema de veredicto actual (a reconsiderar, no es la respuesta)

Esto es lo que el código ya hace hoy con los números de arriba. Se documenta para que quien diseñe reglas nuevas sepa qué se probó y con qué intención — no como base a mantener.

A.1 Vetos y avisos (`gates.py`) — cada uno es una regla fija, se evalúan todas siempre

prueba	condición de veto (para el veredicto)	condición de aviso (no veta)
dd_99	<code>dd_pct_99 > scoring.survival_dd_pct</code> (10%)	—
inflation	<code>inflation > scoring.dd_inflation_flag</code> (3.0)	<code>inflation > scoring.dd_inflation_watch</code> (1.5)
net_5 (B)	<code>net_5 ≤ 0</code>	—
pf_5 (B)	<code>pf_5 ≤ scoring.min_pf</code> (1.10)	—
outlier (B)	—	<code>share > family_b.outlier_frac</code> (0.25)
oos_red/oos_amber (B)	—	<code>oos_ratio < family_b.oos_red_frac</code> (0.40) roja, <code>< oos_amber_frac</code> (0.60) ámbar
Familia C, skip/fill_degrade	<code>median_net ≤ 0 o keep < family_c.skip_keep_frac/fill_keep_frac</code> (0.70/0.75)	—
Familia C, cost_shock/spread_widen	<code>net_5 ≤ 0 o pf_5 ≤ scoring.min_pf</code>	—
high_vol (D)	<code>median_net del tercil alto ≤ 0</code>	—
dead_block (D)	algún bloque no solapado con <code>median_net < 0</code>	—
windows (D)	—	fracción de ventanas móviles con <code>net_5 > 0 < family_d.window_pass_frac</code> (0.70)
concentration (D)	—	<code>concentration > family_d.regime_concentration</code> (0.80)
psr (E)	<code>psr < family_e.psr_gate</code> (0.90)	<code>psr < family_e.psr_target</code> (0.95)
cost_file	—	<code>costs.crosscheck diverge</code> (sección 1.1)
sample	el peor nivel de confianza (sección 9) es "unreliable"	—

A.2 Sub-notas por familia, 0 a 100 (`scoring.py`: `subscores`)

`curve(valor, malo, bueno) = recorta_a[0,100]( 100 × (valor-malo)/(bueno-malo) )` — línea recta entre el valor que da 0 y el que da 100.

```
A = media( curve(inflation, dd_inflation_flag=3.0, 1.0),
curve(survival_dd_pct / dd_pct_99, 1.0, 3.0),
curve(ret_dd al percentil 5 del modelo de cabecera, 0.0, 3.0) )
```

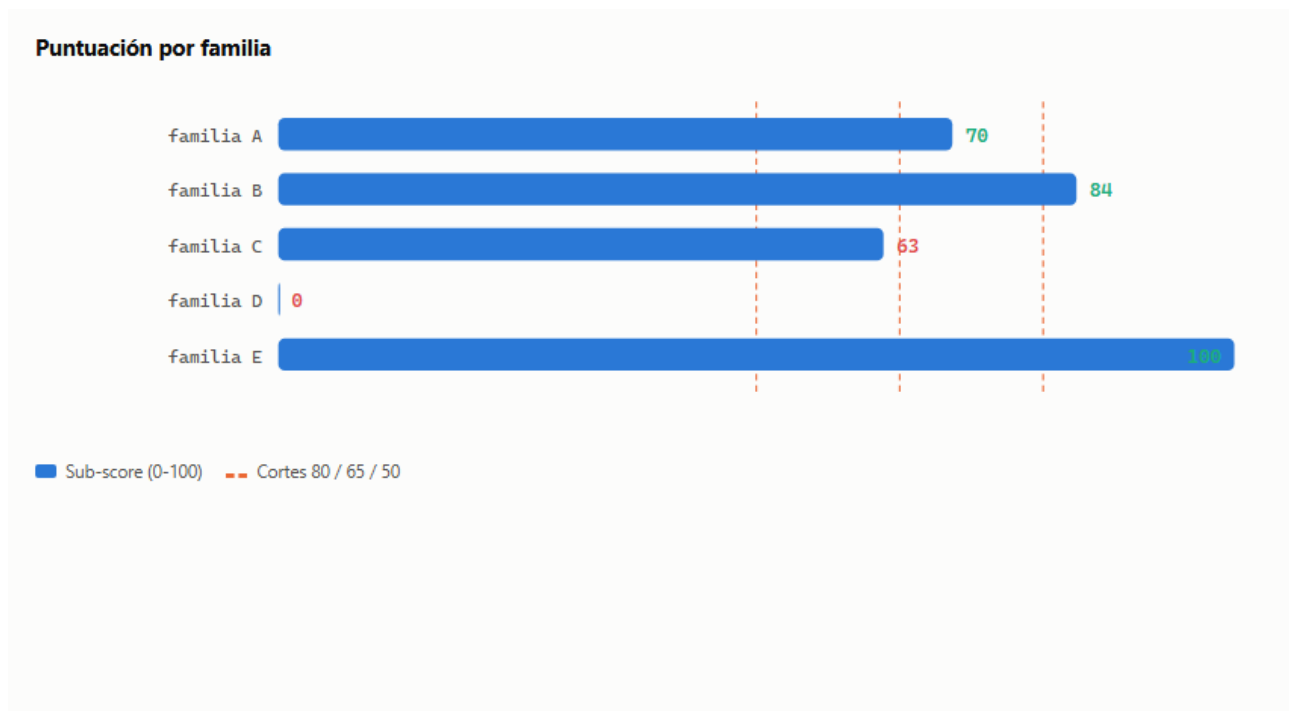
```
B = media( curve(net_5 / net_observado, 0.0, 0.5),
curve(pf_5, min_pf=1.10, 1.5),
curve(outlier_share, outlier_frac=0.25, 0.0) )
```

C = mínimo de `curve(keep, límite de esa prueba, 1.0)` para `skip/fill_degrade`,
y `curve(pf_5, min_pf, 1.5)` para `cost_shock/spread_widen` – el mínimo de las 4, nunca la media

D = mínimo(`curve(fracción de ventanas móviles en positivo, window_pass_frac=0.70, 1.0)`,
`curve(pf_5 mínimo de los bloques no solapados, 1.0, 1.4)`,
`curve(pf_5 del tercil de volatilidad alta, 1.0, 1.4)`)

```
E = curve(psr, psr_gate=0.90, psr_target=0.95)
```

Así se enseñan los cinco sub-scores en el informe, contra los cortes STRONG/ACCEPTABLE/MARGINAL:



A.3 Compuesto y veredicto (scoring.py: verdict)

```
compuesto = A×0.25 + B×0.25 + C×0.20 + D×0.20 + E×0.10 (scoring.weights)
```

si algún veto disparó:

`veredicto = "INCONCLUSIVE"` si TODOS los vetos son de tipo "sample" (poca muestra)

`veredicto = "FAIL"` en cualquier otro caso

si no:

`veredicto = "STRONG"` si `compuesto ≥ 80` (`scoring.tiers[0]`)

`veredicto = "ACCEPTABLE"` si `compuesto ≥ 65` (`scoring.tiers[1]`)

`veredicto = "MARGINAL"` si `compuesto ≥ 50` (`scoring.tiers[2]`)

`veredicto = "FAIL"` en caso contrario

Apéndice B — Distribución observada en un databank real

Estas cifras son un solo databank, un solo activo, una sola fecha — un punto de referencia, no una verdad general. Vienen de las 36 estrategias del databank Results, proyecto XAUUSD, export del 2026-09-03, analizadas el 2026-09-12 con n_sims=100000. Los números de Familia D concretos (windows_ok, high_vol_net, stitched_dd_pct) se calcularon con la configuración de esa fecha: window_sims=2000, window_step_months=3, remuestreo iid_bootstrap dentro de cada ventana, y el camino cosido a un único percentil fijo (5%) — antes de los tres cambios de esta sesión (window_sims=20000, window_step_months=6, stationary en vez de iid_bootstrap, y el camino cosido a cuatro severidades). El resto de columnas (A, B, C, E) no depende de ninguno de esos cambios. No se ha vuelto a correr el databank completo con la configuración nueva — sería el primer candidato si se quiere una tabla de calibración actualizada.

métrica	p0 (mín)	p5	p25	mediana	p75	p95	p100 (máx)
compuesto	21.3	23.5	32.8	39.9	46.8	56.6	61.1
sub-score A	19.3	23.3	28.2	41.8	50.9	65.0	70.3
sub-score B	18.4	29.1	49.7	62.4	67.7	78.6	83.6
sub-score C	0.0	0.0	11.9	21.7	35.5	55.3	63.3
sub-score D	0.0	0.0	0.0	0.0	0.0	0.0	0.0
sub-score E	100.0	100.0	100.0	100.0	100.0	100.0	100.0
operaciones	982	1058	1222	1329	1480	1734	1936
beneficio neto real (\$)	25 994	27 680	38 401	43 730	52 218	60 021	70 141
drawdown real (%)	2.1	2.2	3.5	4.9	6.3	9.4	12.6
Ret/DD real	1.88	2.41	4.74	6.92	9.77	14.47	15.47
drawdown reordenado p95 (%)	4.2	4.4	7.1	9.7	13.1	17.3	20.4
drawdown reordenado p99 (%)	5.2	5.4	9.1	12.6	17.1	22.9	27.0
inflación del drawdown (x)	1.53	1.61	1.82	1.95	2.18	2.61	2.76
beneficio remuestreado p5 (\$)	905	4 065	13 335	21 773	28 640	32 934	42 730
profit factor remuestreado p5	1.005	1.018	1.083	1.139	1.191	1.307	1.363
Sharpe OOS / Sharpe IS	−0.51	−0.38	−0.02	0.14	0.27	0.58	0.75
share de la mejor operación	2.8%	3.6%	4.0%	4.9%	6.3%	9.2%	12.8%

métrica	p0 (mín)	p5	p25	mediana	p75	p95	p100 (máx)
ventanas móviles en positivo	9.6%	15.4%	25.0%	31.7%	38.5%	48.1%	48.1%
beneficio remuestreado, tercil alto (\$)	-2 200	2 030	6 982	10 132	14 414	20 632	22 046
drawdown del peor camino posible (%)	8.5	10.1	17.1	20.4	29.0	40.7	44.3
PSR	0.972	0.977	0.997	1.000	1.000	1.000	1.000

Veredictos reales de ese databank: 33 de 36 FAIL, 3 MARGINAL, 0 STRONG/ACCEPTABLE/ INCONCLUSIVE.

Dos hechos crudos, sin interpretar: **el sub-score D es 0.0 en las 36 estrategias sin excepción**, y **el sub-score E es 100.0 en las 36 sin excepción**. Ninguna de las dos cosas se explica en este documento — son datos, no un diagnóstico.

Apéndice C — config.yaml completo, con el significado de cada valor

```
global:
starting_equity: 100000.0 # cuenta desde la que arranca la curva de equity aditiva
risk_per_trade: 1000.0 # USD arriesgados por operación; el divisor que convierte el P&L en R
n_sims: 100000 # rutas simuladas por cada sub-prueba
chunk: 2000 # rutas por lote de cada worker; memoria, no estadística
max_workers: null # núcleos en paralelo; null = todos los que tenga la máquina
percentile_set: [1, 5, 25, 50, 75, 95, 99] # percentiles que se guardan de cada distribución
report_percentile: 95 # el percentil contra el que se compara el valor observado, y el que marcan los histogramas
progress_update_pct: 1 # cada cuánto se refresca la barra de progreso, en %

blocks:
block_min: 5 # bloque más pequeño de operaciones consecutivas que se mantiene junto
min_blocks: 20 # un reordenamiento necesita al menos estos bloques para aleatorizar algo
n_block_sizes: 6 # tamaños de bloque probados, repartidos entre block_min y N/min_blocks

family_b:
oos_amber_frac: 0.60 # Sharpe mediano OOS por debajo de esta fracción del de IS: aviso
oos_red_frac: 0.40 # ídem, veto
outlier_frac: 0.25 # beneficio neto perdido al quitar la mejor operación, umbral de aviso

family_c:
p_skip: 0.05
skip_keep_frac: 0.70
cost_shock_range: [1.0, 2.0]
fill_frac: 0.15
fill_depth: 0.25
fill_keep_frac: 0.75
spread_scale: [1.0, 2.0]
cost_crosscheck: true

family_d:
window_months: 24
window_step_months: 6 # cambiado en esta sesión (era 3)
window_pass_frac: 0.70
window_sims: 20000 # cambiado en esta sesión (era 2000)
vol_model: atr # atr | garch
```

```

atr_period: 20
garch_order: [1, 1]
garch_dist: t
regime_concentration: 0.80
stitch_quantiles: [0.01, 0.05, 0.10, 0.25] # nuevo en esta sesión; severidad del camino cosido, ninguna
es "la" respuesta

family_e:
psr_benchmark: 0.0
psr_gate: 0.90
psr_target: 0.95

scoring:
dd_inflation_flag: 3.0
dd_inflation_watch: 1.5
survival_dd_pct: 0.10 # PROVISIONAL hasta que se conozcan las reglas reales de la prop firm
min_pf: 1.10
weights: {A: 0.25, B: 0.25, C: 0.20, D: 0.20, E: 0.10}
tiers: [80, 65, 50] # cortes STRONG / ACCEPTABLE / MARGINAL

stability:
n_stability_runs: 8
stability_tol: 0.10

```

Coste del activo (assets/XAUUSD.yaml, valores sqx_default de la última vez que se generó este documento):
point_value = 100.0 USD por 1.0 de precio por lote, tick_size = 0.01, spread = 10.0 puntos, commission = 8.0 USD por lote y lado.

Apéndice D — Glosario mínimo

- **IS / OOS:** in-sample (periodo con el que SQX construyó la estrategia) / out-of-sample (periodo que no vio). La etiqueta viene de SQX, no se recalcula aquí.
 - **Ruta simulada:** una de las `n_sims` reordenaciones/remuestreos/estrés de las mismas operaciones.
 - **Backtest / observado:** el único resultado real, el que SQX produjo con las operaciones en su orden verdadero — el punto fijo contra el que se compara cada distribución simulada.
 - **Percentil q de una distribución:** el valor por debajo del cual cae el q% de las rutas simuladas.
 - **Rank:** en qué percentil de la distribución simulada cae el valor del backtest — no un percentil fijo de la config, sino "dónde queda lo que pasó de verdad".
 - **Veto vs. aviso:** un veto obliga al veredicto a FAIL (o INCONCLUSIVE); un aviso queda registrado pero no cambia el veredicto por sí solo.
-

Apéndice E — El informe completo, renderizado

Las páginas que siguen son el informe HTML real de Strategy 17.18.29 (el mismo databank y fecha de todo este documento, `n_sims=100000`), impreso a PDF tal cual lo renderiza un navegador — `strategies/monteCarlo/report.py` escribe exactamente este archivo en `estrategias/Strategy 17.18.29.html` cuando corre el databank entero, y el panel interactivo (`explorer/serve.py`) construye la misma página sección por sección bajo demanda. Todo lo descrito en las secciones 1 a 10 de este documento está aquí, en el orden en que un lector — o un agente con un navegador — se lo encontraría: veredicto primero, qué falló, luego cada familia con sus tablas y sus figuras.

Strategy 17.18.29

Robustez de una estrategia ya aceptada: cuánto de este resultado es suerte, y de qué tipo.

MARGINAL

Veredicto

61

Compuesto sobre 100

0

Vetos disparados

982

Operaciones

Ninguna prueba veta. Lo que limita la nota es la familia D (0 sobre 100): ventanas que aguantan, el peor bloque de 24 meses, y el tercil de volatilidad alta. El compuesto es 61.

Lo que falló

- aviso · familia A · Drawdown inflado (aviso)

— El drawdown reordenado es 1.7 veces el del backtest; por encima de 1.5 conviene vigilarlo, no descartarlo.
- aviso · familia B · Caída OOS

— El Sharpe mediano fuera de muestra es el 35% del de dentro. Eso es optimismo que el test de decaimiento no recogió.
- aviso · familia D · Ventanas móviles

— Sólo el 50% de las ventanas móviles aguanta en positivo en su percentil 5 (se pide 70%).

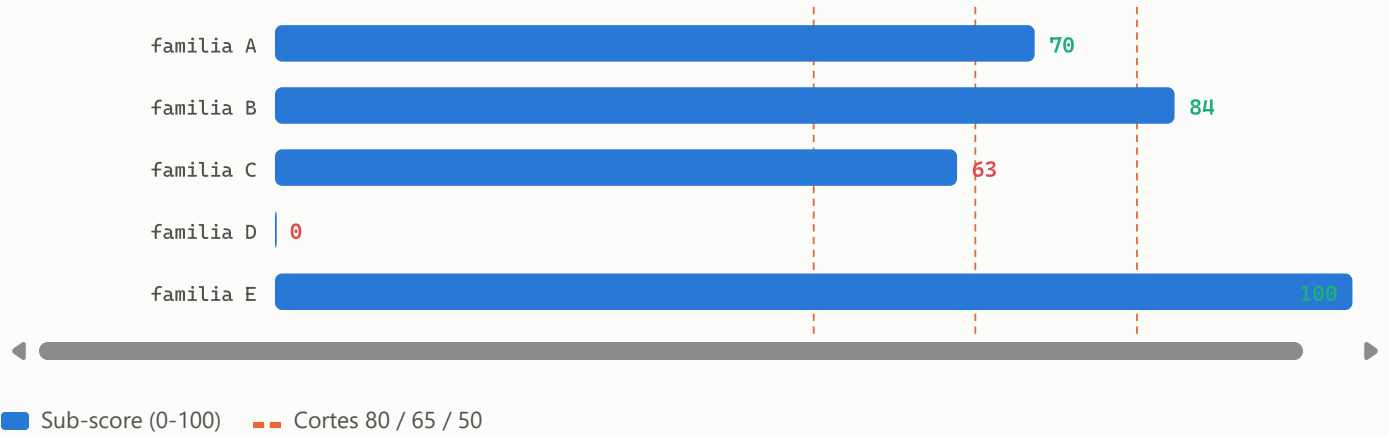
De un vistazo

El backtest contra la mediana de las simulaciones. La columna «rango» dice qué fracción de las simulaciones quedó por debajo del backtest: cerca del 100% es un backtest afortunado.

	BACKTEST	MEDIANA SIMULADA	PERCENTIL 5	RANGO DEL BACKTEST	CONFIAN
Beneficio neto	42,837 \$	42,843 \$	28,835 \$	50%	reliab
Sharpe por operación	0.16	0.16	0.11	49%	reliab
Drawdown máximo	2.46%	2.68%	1.72%	38%	reliab
Ret/DD	12.78	13.16	6.24	47%	reliab
Racha perdedora	8.00	9.00	7.00	48%	reliab
Inflación del drawdown	1.69×	—	—	—	reliab

PSR	1.0000	-	-	-	reliab
-----	--------	---	---	---	--------

Puntuación por familia



- **A** — inflación del drawdown, margen del percentil 99 frente al techo de cuenta, y el Ret/DD del percentil 5
- **B** — beneficio y profit factor del percentil 5, y cuánto pesa la mejor operación
- **C** — la peor de las cuatro pruebas de ejecución
- **D** — ventanas que aguantan, el peor bloque de 24 meses, y el tercil de volatilidad alta
- **E** — la Probabilistic Sharpe Ratio

Familia A — suerte de orden

70

sin vetos

Sub-score familia A de esta familia

- **Aviso · Drawdown inflado (aviso)** — El drawdown reordenado es 1.7 veces el del backtest; por encima de 1.5 conviene vigilarlo, no descartarlo.

Las mismas operaciones, en otro orden. El beneficio no cambia; el drawdown y las rachas sí. Lo que mide es cuánto del drawdown cómodo del backtest fue el orden en que llegaron.

MODELO (PERCENTIL 95)	BENEFICIO NETO	RETORNO	DRAWDOWN \$	DF
Backtest	42,837 \$	42.84%	3,351 \$	
Barajado i.i.d.	42,837 \$	42.84%	4,956 \$	
Bloques barajados — bloques de 5 operaciones	42,837 \$	42.84%	4,938 \$	

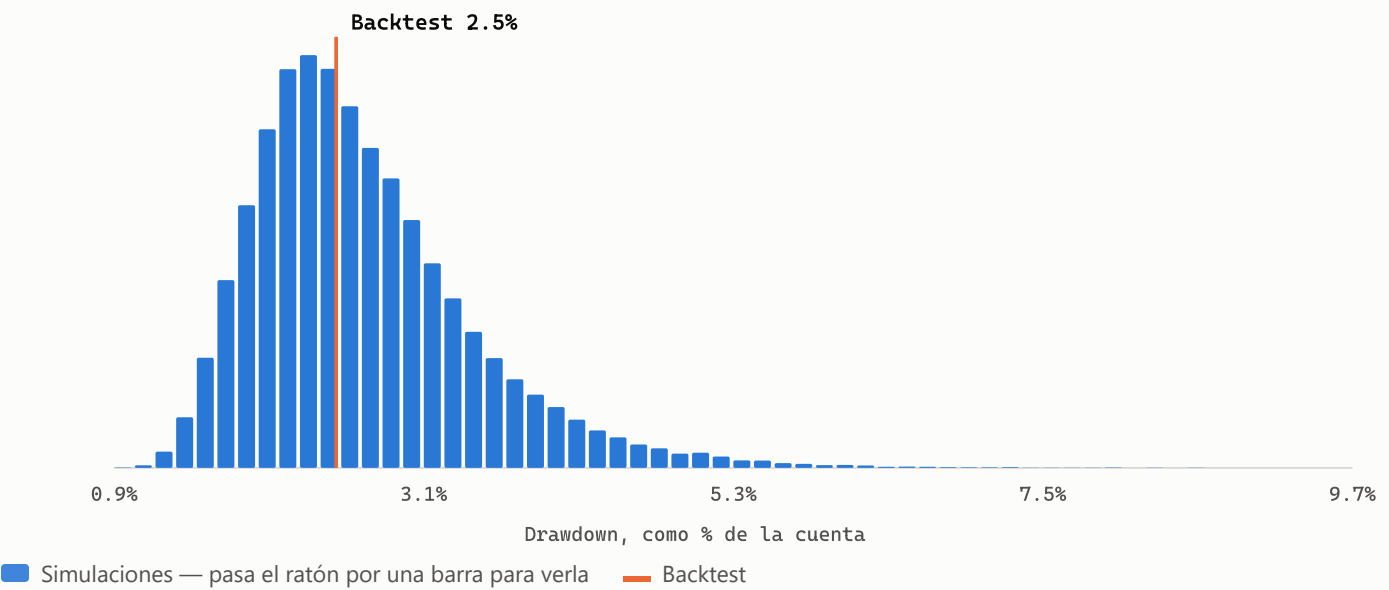
Bloques barajados — bloques de 14 operaciones	42,837 \$	42.84%	4,302 \$
Bloques barajados — bloques de 23 operaciones	42,837 \$	42.84%	3,599 \$
Bloques barajados — bloques de 31 operaciones	42,837 \$	42.84%	4,412 \$
Bloques barajados — bloques de 40 operaciones	42,837 \$	42.84%	3,650 \$
Bloques barajados — bloques de 49 operaciones	42,837 \$	42.84%	3,728 \$
Bootstrap estacionario — bloques de 10 operaciones de media	56,236 \$	56.24%	4,845 \$

Comprobación: barajar sin reemplazo (i.i.d. y bloques) movió el beneficio neto en $2.19e-11$ \$ — cero, como debe ser: son permutaciones puras, todas las operaciones exactamente una vez, así que su beneficio no puede moverse.

El bootstrap estacionario es distinto por dentro. No es una permutación: dibuja bloques de longitud variable (media geométrica alrededor del tamaño de bloque) que pueden solaparse o dar la vuelta al final de la serie, así que una misma operación puede aparecer más de una vez y otra ninguna. Por eso su beneficio neto, a diferencia de «barajado» y «bloques barajados», **sí** se mueve de simulación en simulación — es el precio de no imponer una única longitud de dependencia, y es también la fila que da el drawdown de cabecera del informe.

Drawdown máximo reordenando

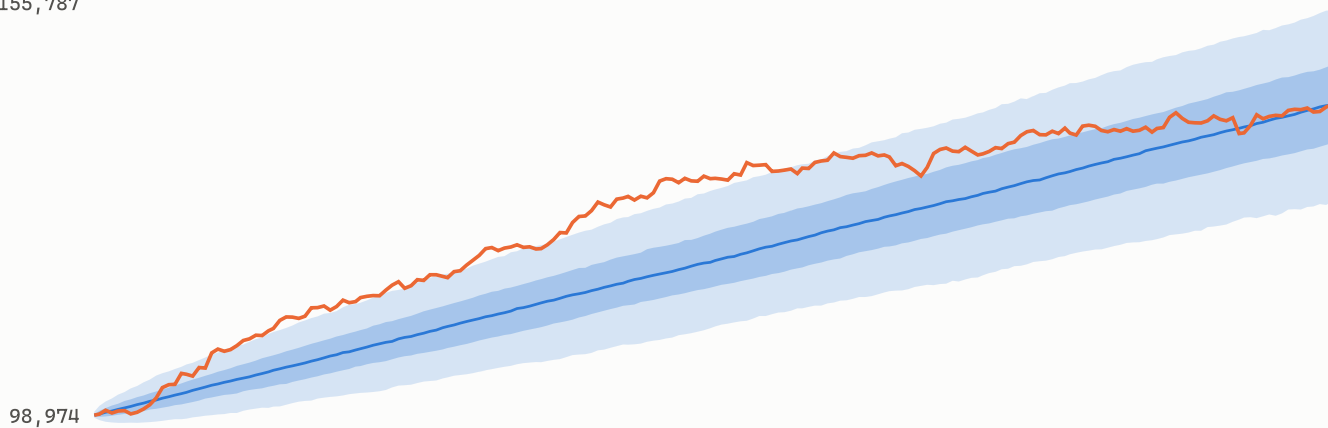
percentil 95 (fijo, no `cfg.report_percentile`): 4.16% · backtest 2.46% · inflación 1.69×



A dónde llegaba la curva en otro orden

bootstrap estacionario — bloques de 10 operaciones de media, 200 puntos

155,787



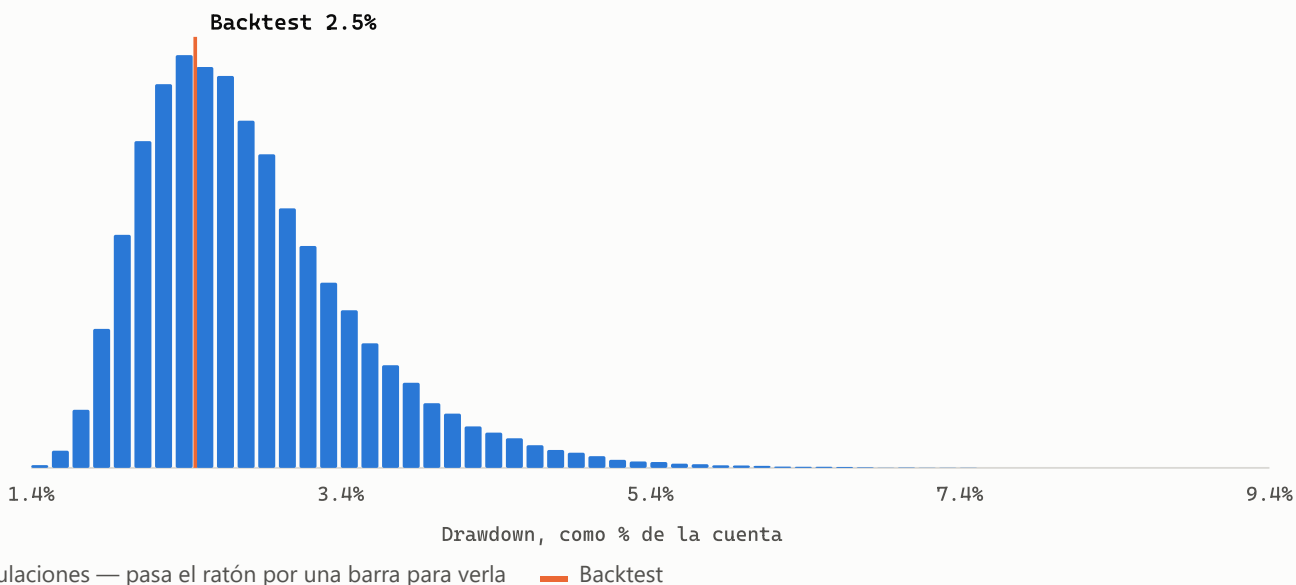
operaciones, en orden

Banda 5-95 Banda 25-75 Mediana simulada Backtest

Cada modelo, por separado

El mismo drawdown reordenado, un histograma por cada tamaño de bloque y cada modelo — para ver si un bloque concreto se comporta distinto, no solo la media de todos.

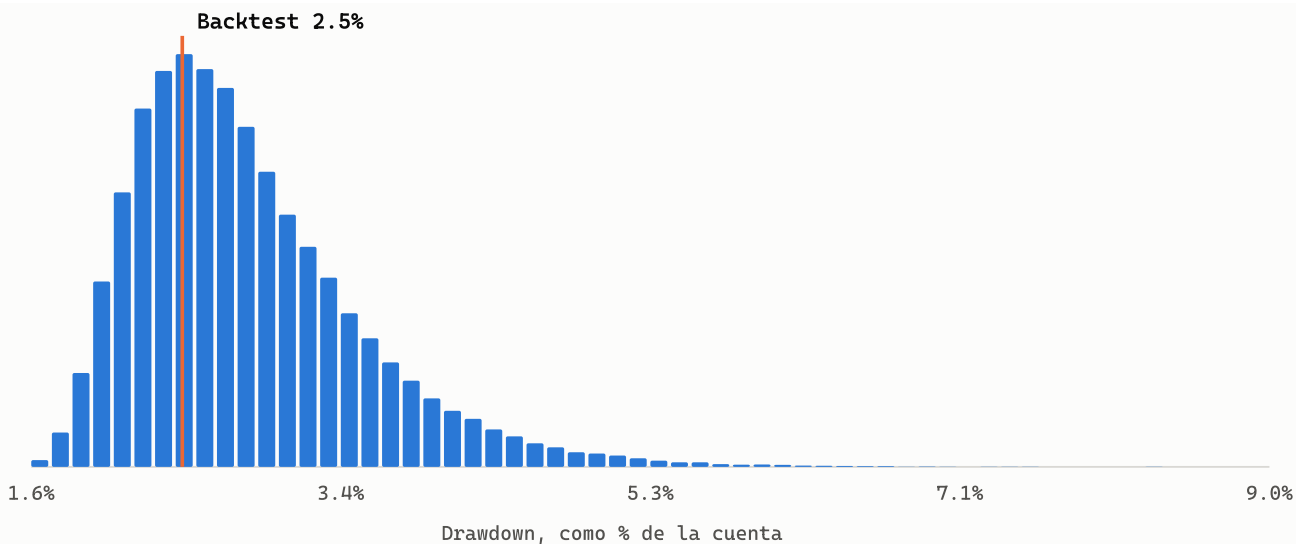
Drawdown — Barajado i.i.d.



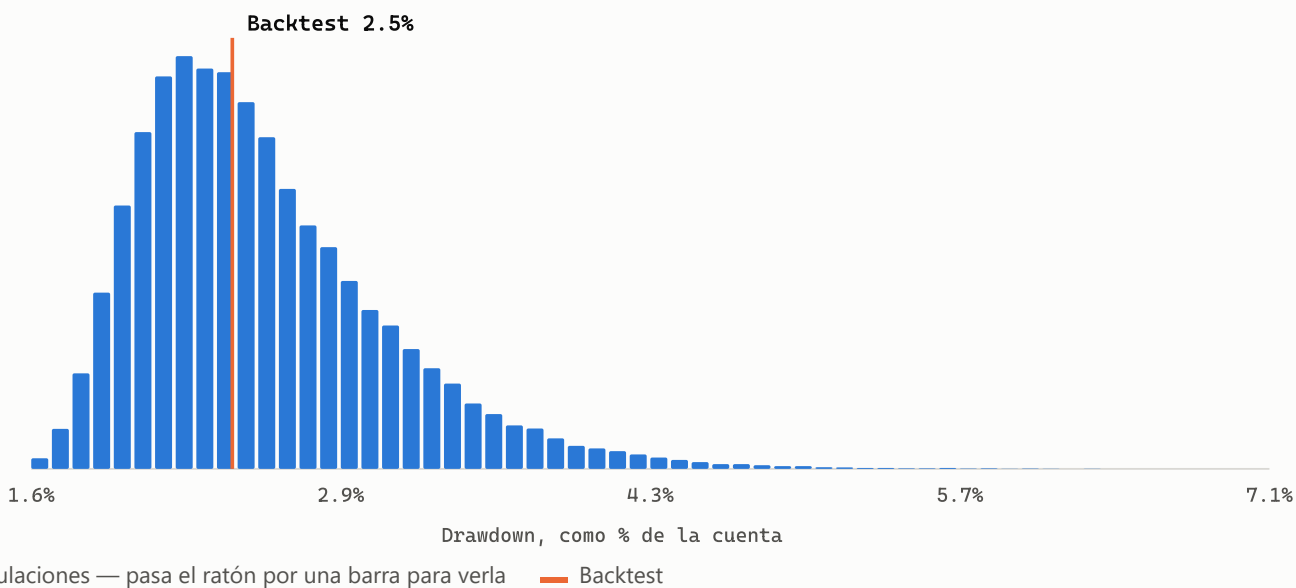
Drawdown, como % de la cuenta

Simulaciones — pasa el ratón por una barra para verla Backtest

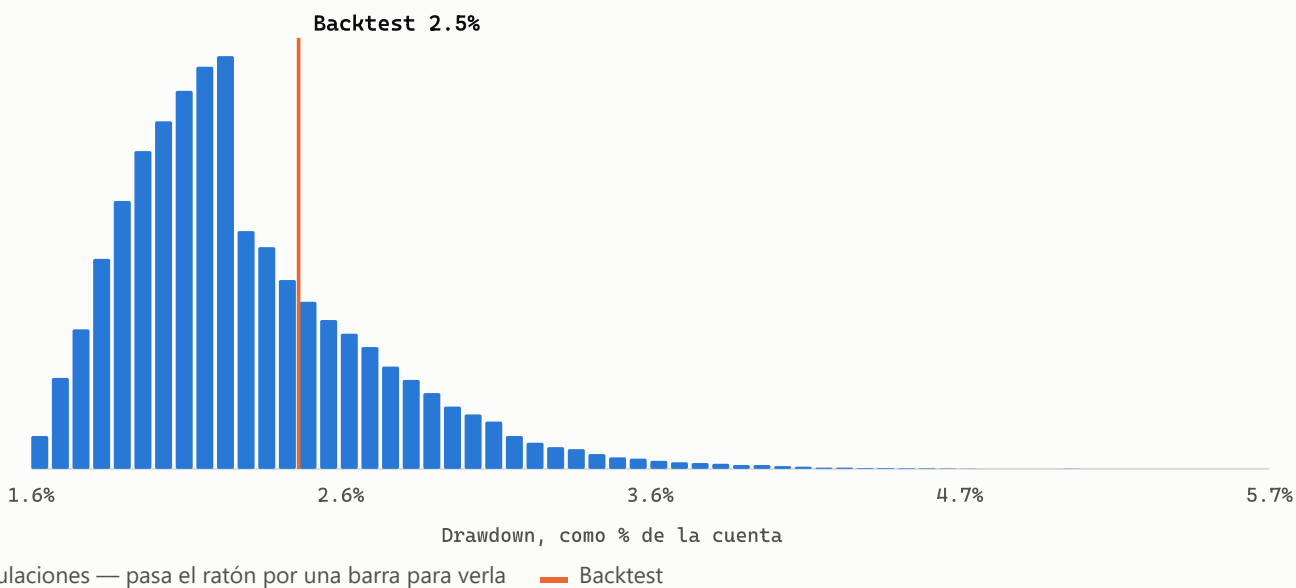
Drawdown — Bloques barajados — bloques de 5 operaciones



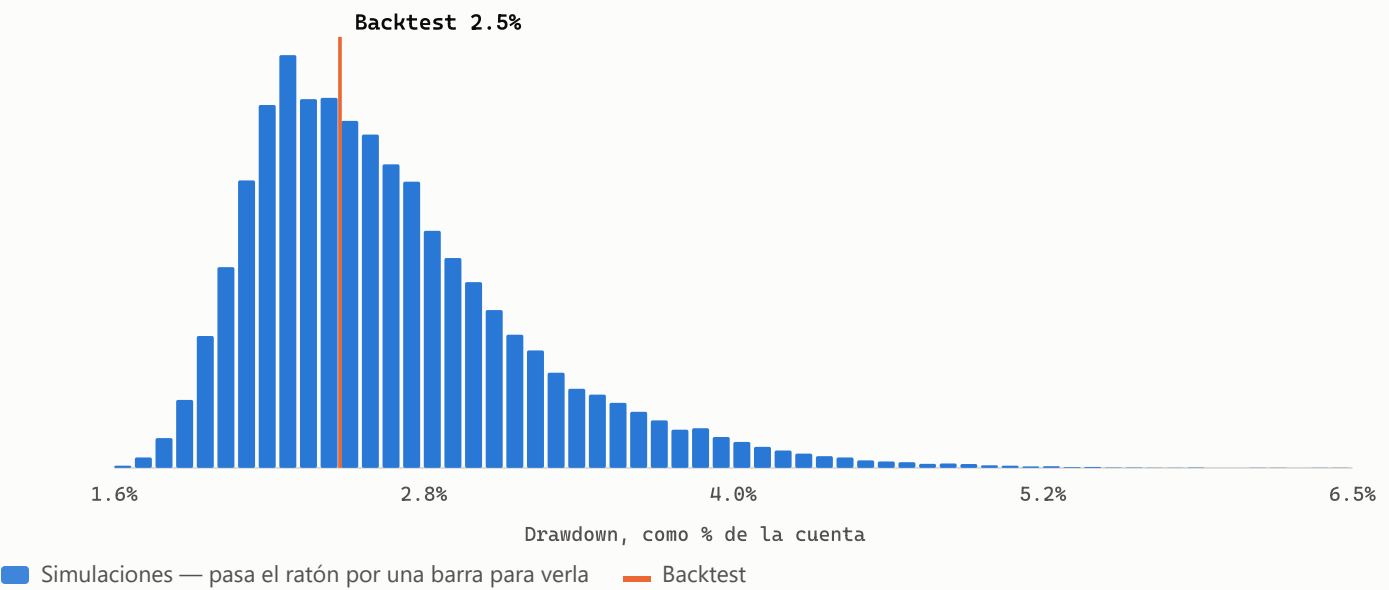
Drawdown — Bloques barajados — bloques de 14 operaciones



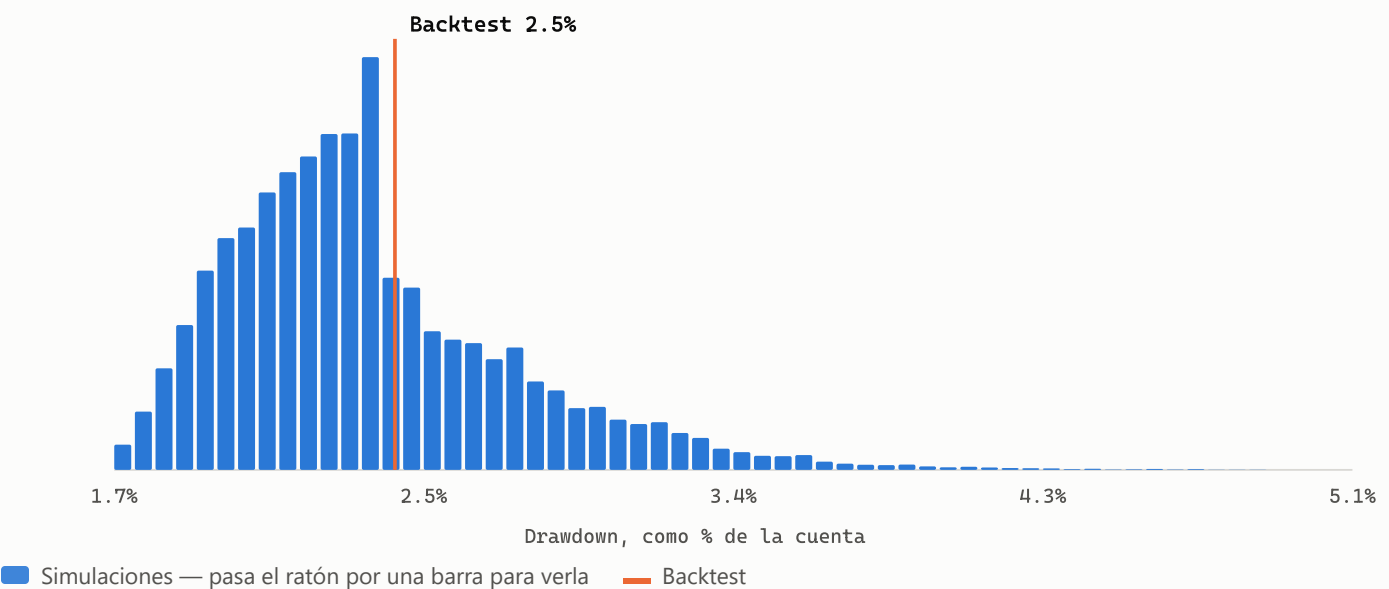
Drawdown — Bloques barajados — bloques de 23 operaciones



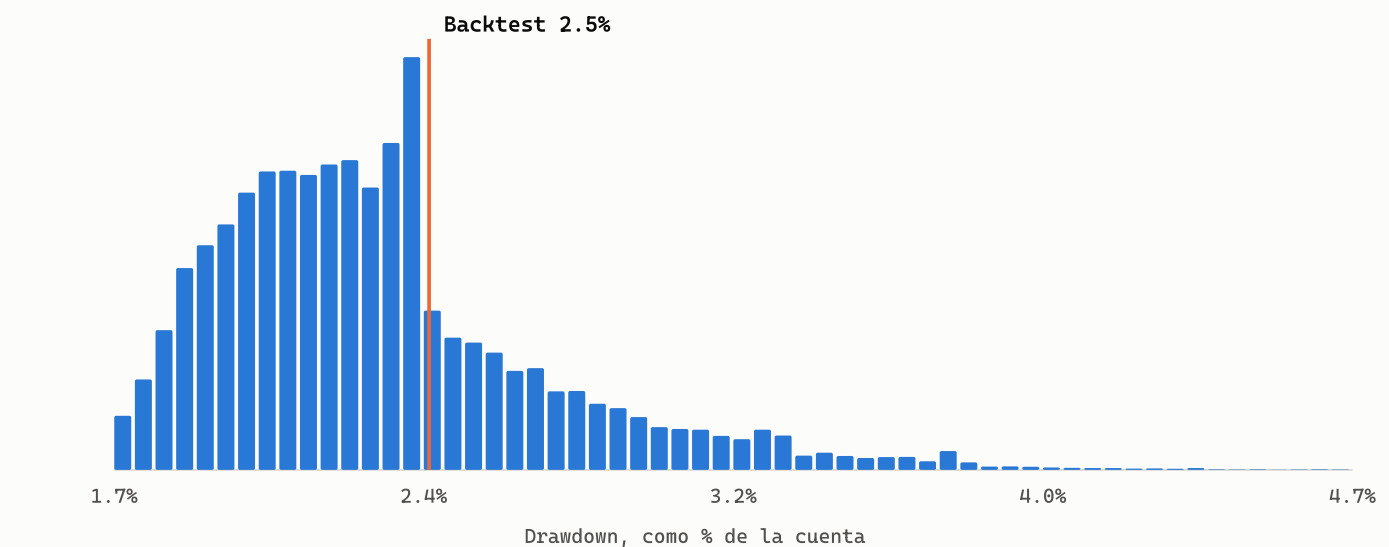
Drawdown — Bloques barajados — bloques de 31 operaciones



Drawdown — Bloques barajados — bloques de 40 operaciones

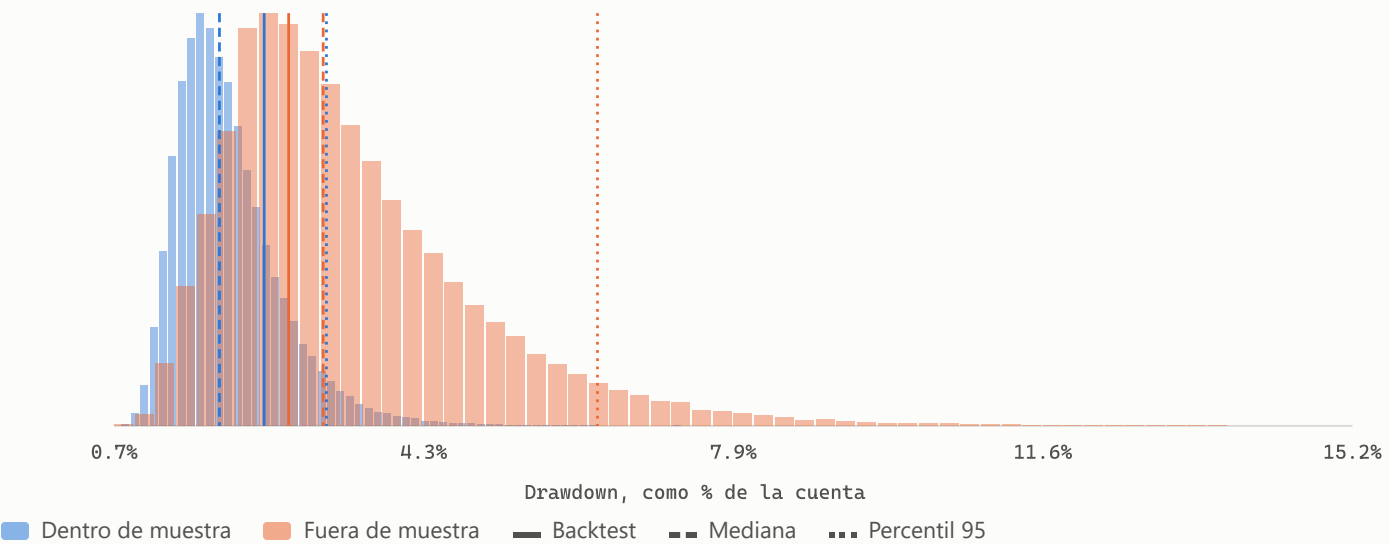


Drawdown — Bloques barajados — bloques de 49 operaciones



Degradación dentro / fuera de muestra

Drawdown reordenando — IS vs OOS



MUESTRA	OPERACIONES	BACKTEST	MEDIANA	PERCENTIL 95
Dentro de muestra	690	2.5%	1.9%	3.2%
Fuera de muestra	292	2.7%	3.1%	6.4%

Familia B — suerte de composición

84

sin vetos

Sub-score familia B de esta familia

- Aviso · Caída OOS** — El Sharpe mediano fuera de muestra es el 35% del de dentro. Eso es optimismo que el test de decaimiento no recogió.

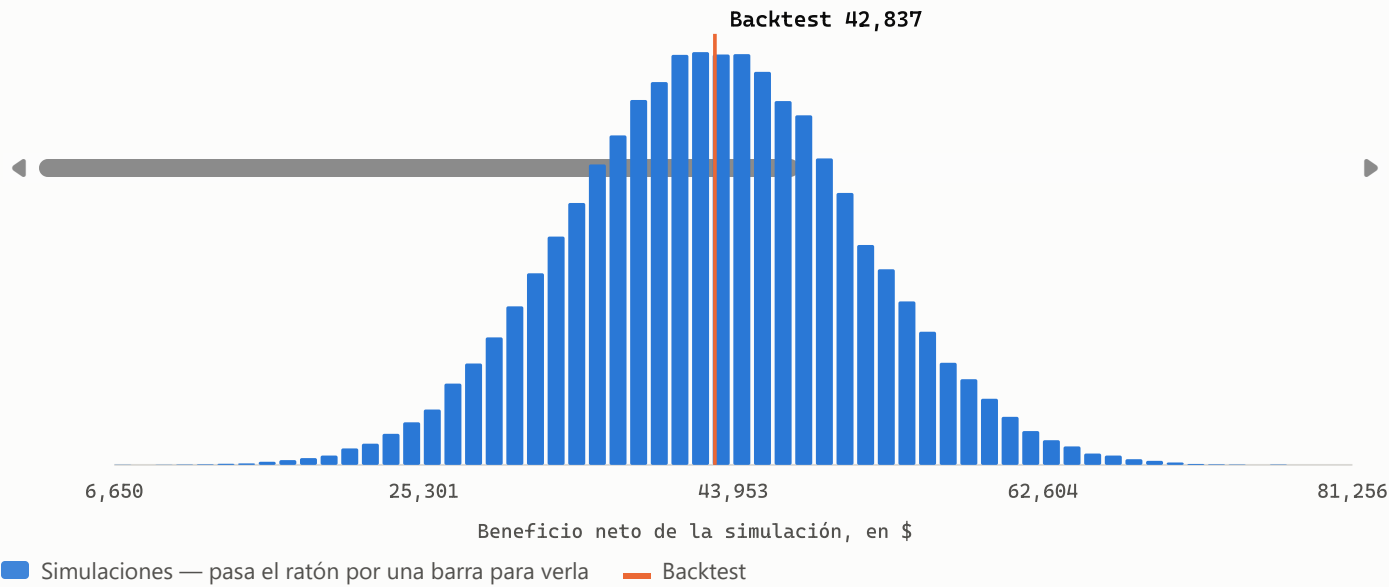
Se vuelven a sortear las operaciones con reemplazo: aquí sí cambia el beneficio.
Responde a cuánto del resultado descansa en unas pocas operaciones concretas.

MODELO (PERCENTIL 5)	BENEFICIO NETO	RETORNO	DRAWDOWN \$	DRAWDOWN
Remuestreo i.i.d.	28,835 \$	28.83%	2,130 \$	1.72
Bloques remuestreados — bloques de 5 operaciones	28,907 \$	28.91%	2,127 \$	1.72
Bloques remuestreados — bloques de 14 operaciones	29,650 \$	29.65%	2,026 \$	1.64

Bloques remuestreados — bloques de 23 operaciones	29,989 \$	29.99%	1,994 \$	1.62
Bloques remuestreados — bloques de 31 operaciones	29,968 \$	29.97%	1,983 \$	1.62
Bloques remuestreados — bloques de 40 operaciones	29,982 \$	29.98%	1,944 \$	1.62
Bloques remuestreados — bloques de 49 operaciones	30,070 \$	30.07%	1,917 \$	1.60

Beneficio neto remuestreando

percentil 5: 28,835 \$ · PF percentil 5 1.36

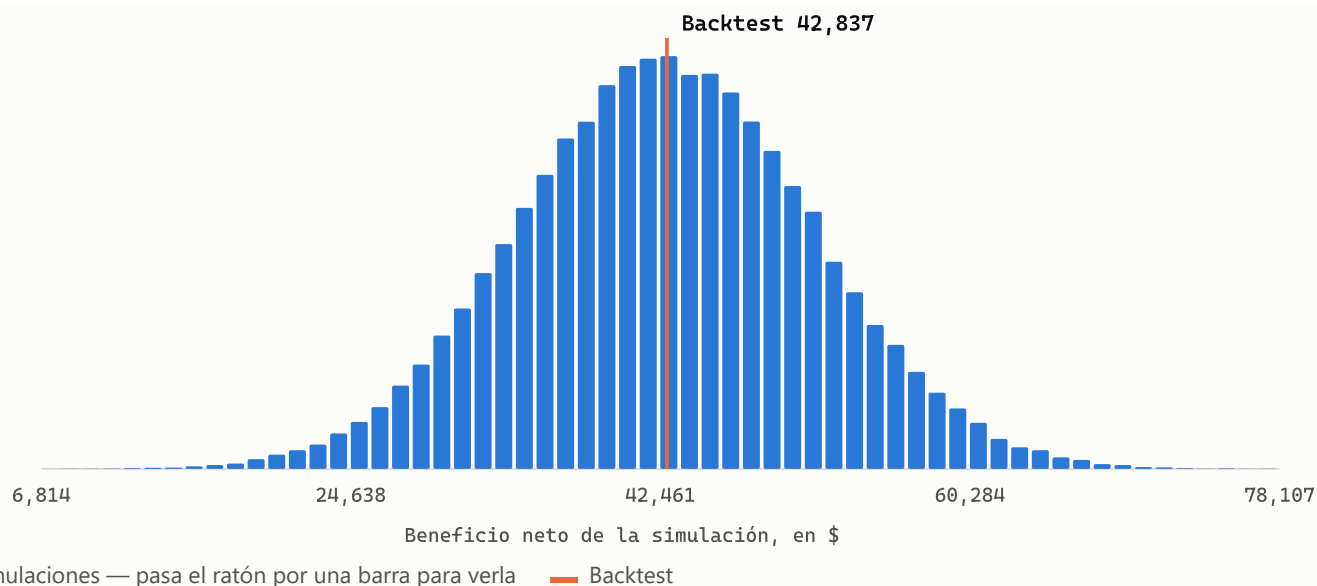


Dependencia de la mejor operación. Quitándola, el beneficio baja de 42,837 \$ a 41,218 \$ — el 4% del total.

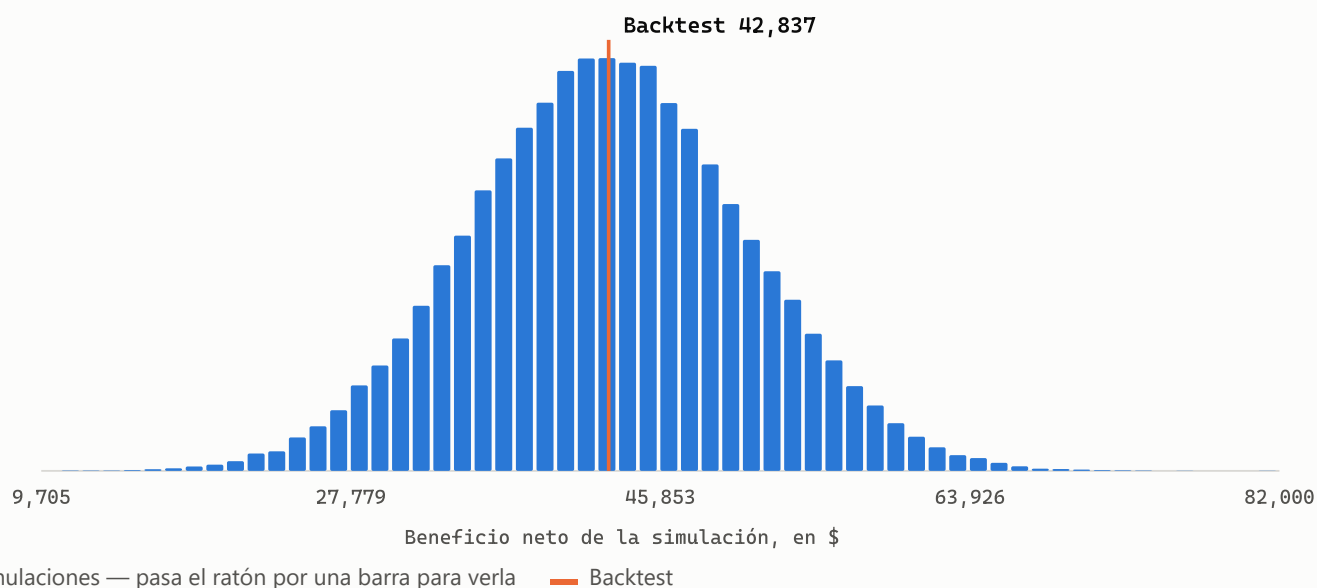
Cada modelo, por separado

El mismo beneficio remuestreado, un histograma por cada tamaño de bloque — el i.i.d. no tiene bloque, así que su composición cambia de la forma más agresiva posible; los de bloque grande se acercan más al backtest real.

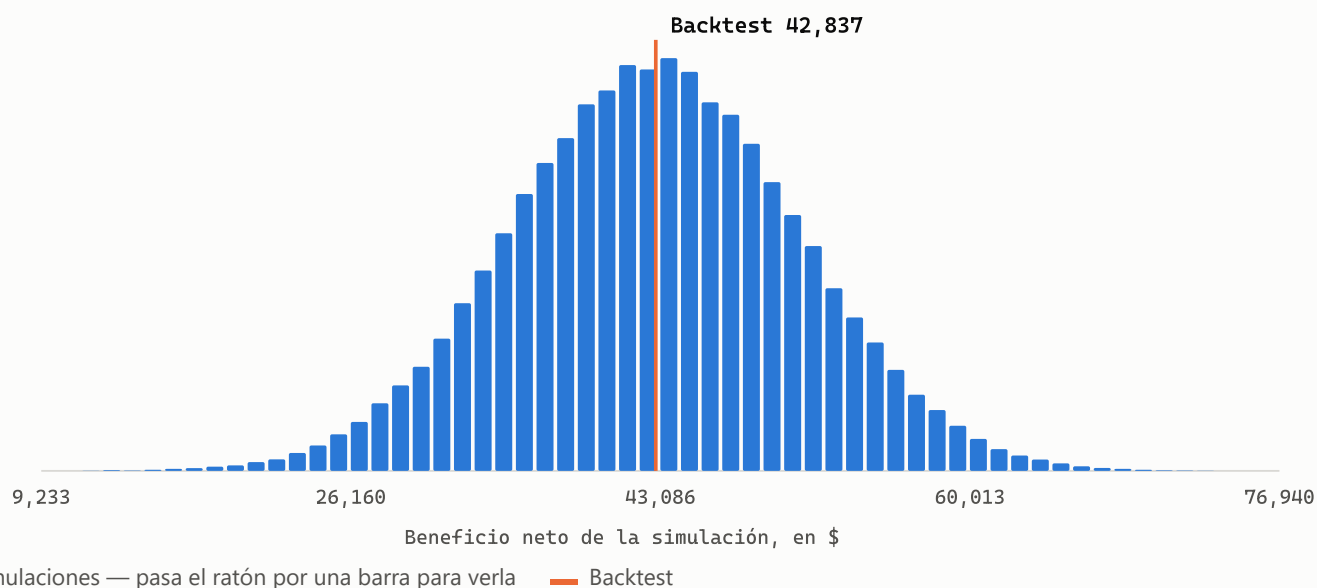
Beneficio — Bloques remuestreados — bloques de 5 operaciones



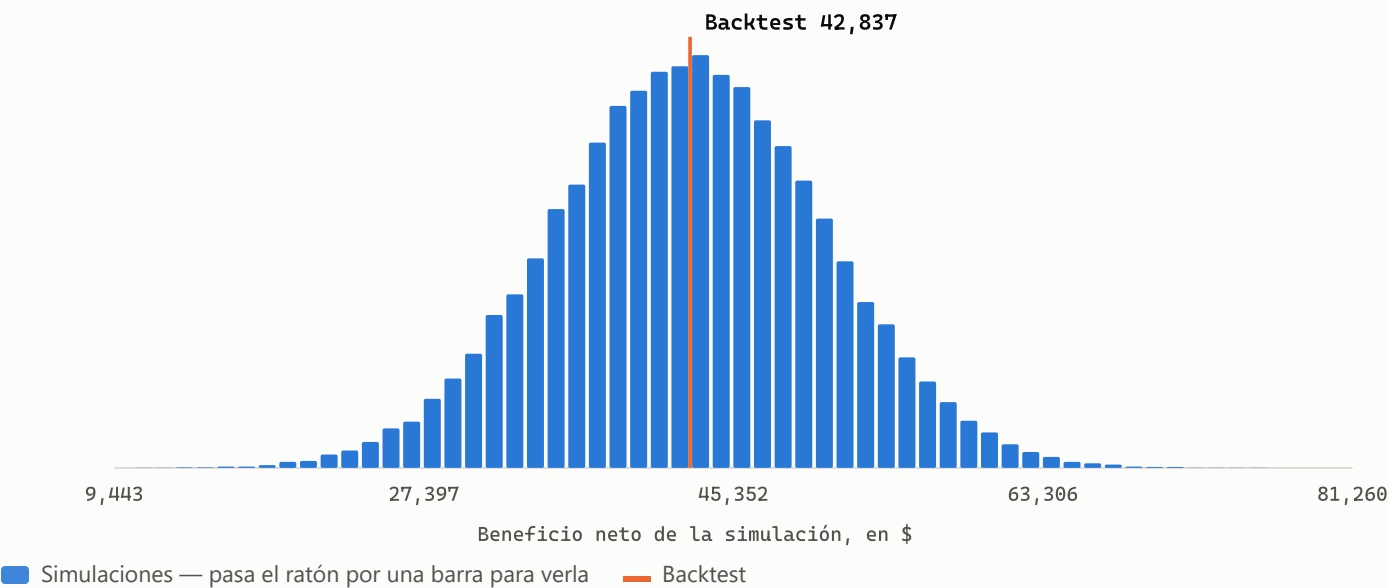
Beneficio — Bloques remuestreados — bloques de 14 operaciones



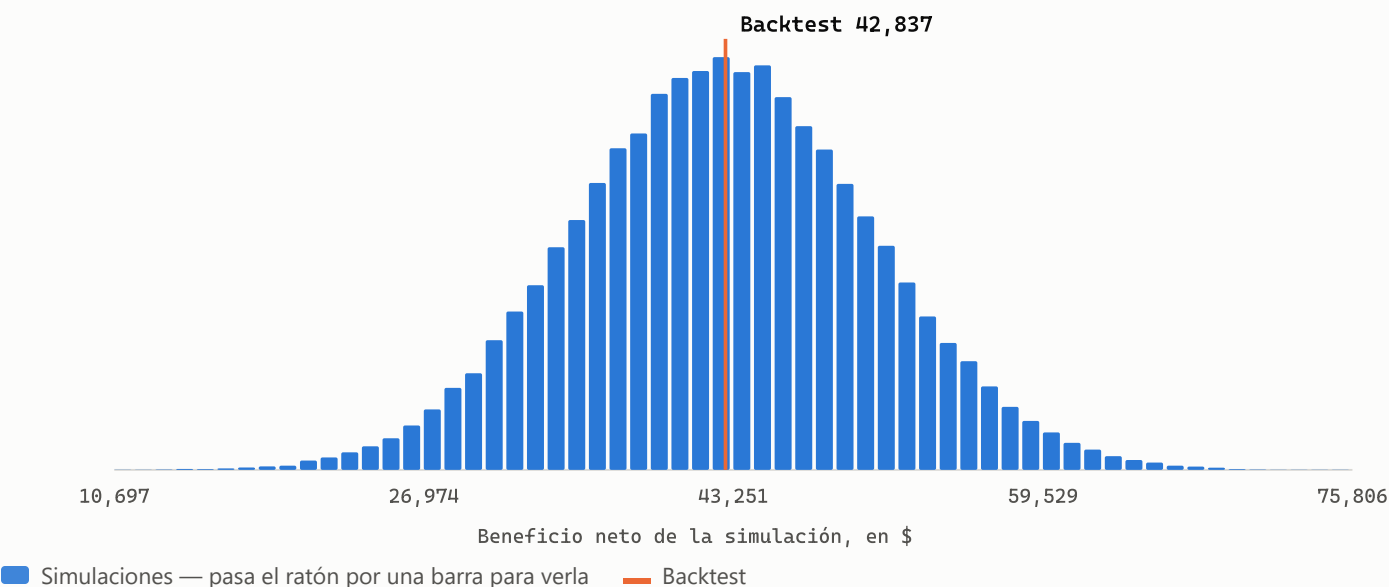
Beneficio — Bloques remuestreados — bloques de 23 operaciones



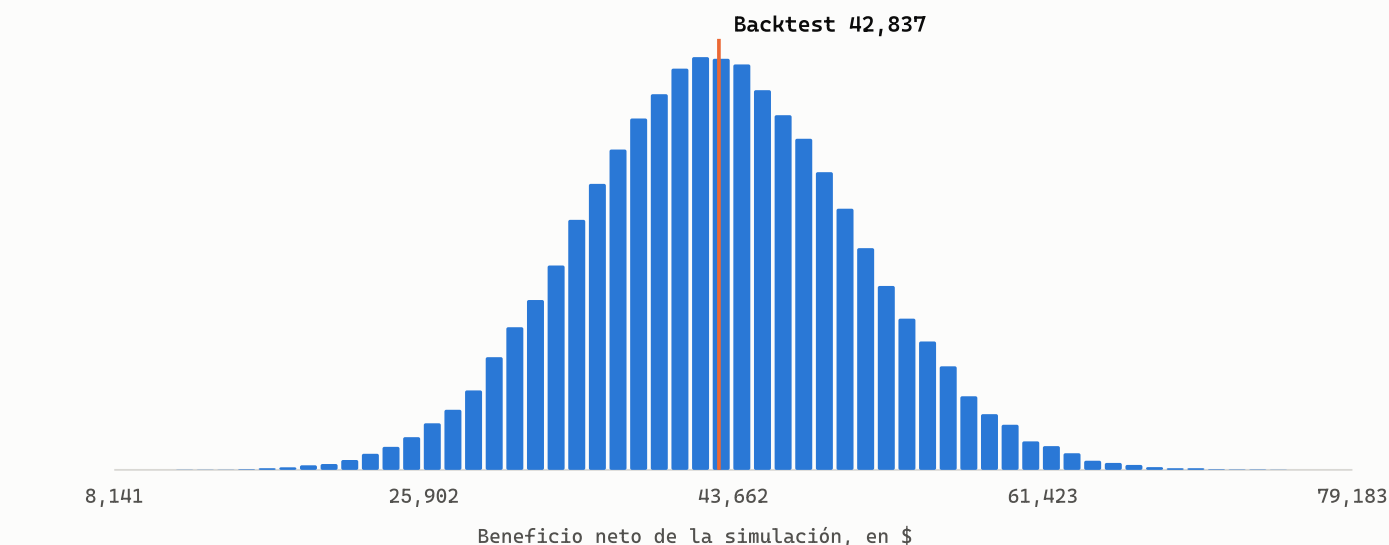
Beneficio — Bloques remuestreados — bloques de 31 operaciones



Beneficio — Bloques remuestreados — bloques de 40 operaciones



Beneficio — Bloques remuestreados — bloques de 49 operaciones



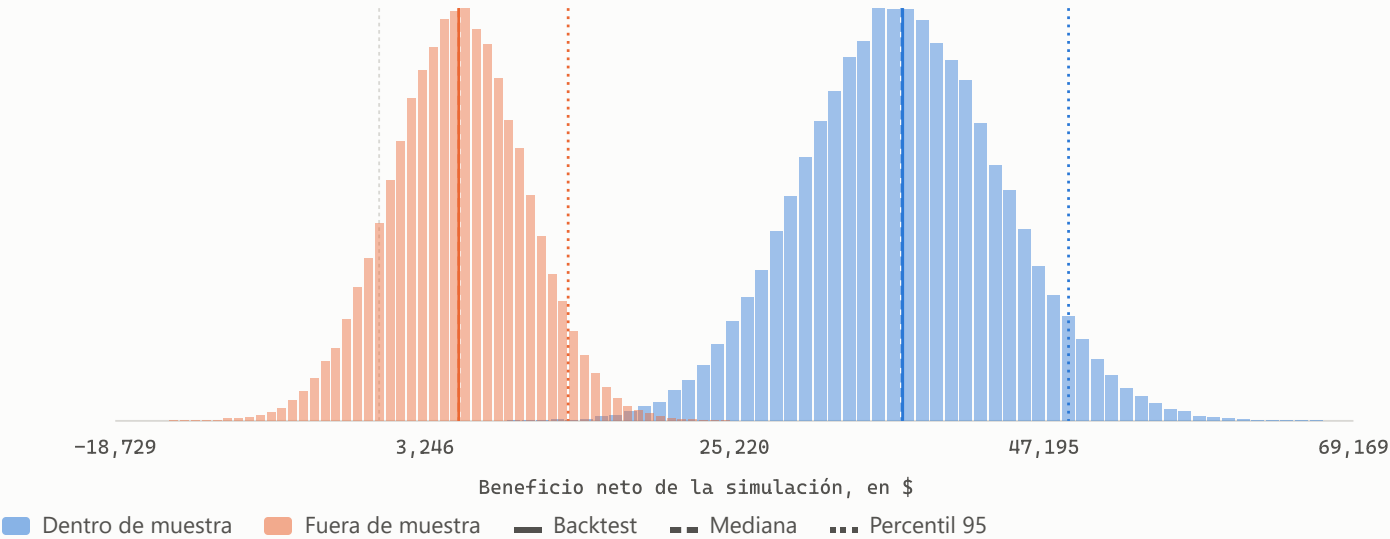
Dentro y fuera de muestra

No detecta sobreajuste: compara el nivel. Una caída grande del Sharpe mediano al pasar a OOS es optimismo que el test de decaimiento no recogió.

MUESTRA	OPERACIONES	SHARPE MEDIANO	BENEFICIO MEDIANO	PF P5	CONFIANZA
IS	690	0.1999	37,098 \$	1.49	reliable
OOS	292	0.0708	5,678 \$	0.93	reliable

El Sharpe fuera de muestra es el **35%** del de dentro.

Beneficio remuestreado — IS vs OOS



MUESTRA	OPERACIONES	BACKTEST	MEDIANA	PERCENTIL 95
Dentro de muestra	690	3.717e+04	3.711e+04	4.894e+04
Fuera de muestra	292	5,663	5,676	1.342e+04

Familia C — suerte de ejecución

63

Sub-score familia C

sin vetos

de esta familia

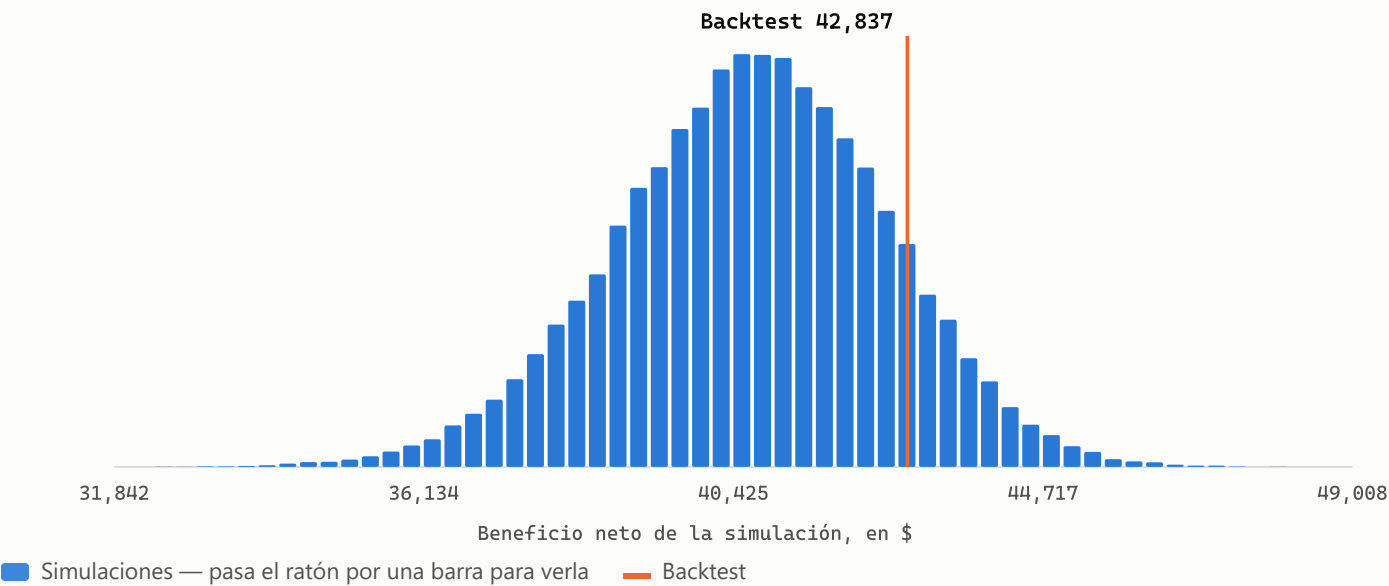
Ninguna prueba de esta familia falló ni avisó.

Las mismas operaciones peor ejecutadas. Las cuatro tienen que pasar: no se promedian, y las cuatro se ejecutan siempre, aunque alguna ya haya fallado — igual que todas las demás pruebas del informe.

- **Entradas perdidas** — Entradas que el sistema real no llega a tomar
- **Costes hasta el doble** — Comisión y swap hasta el doble de lo que SQX cobró
- **Ejecuciones degradadas** — Ejecuciones que devuelven parte de lo que la propia operación ya había cedido
- **Spread más ancho** — Un spread más ancho que el fijo que supuso el backtest

PRUEBA	BENEFICIO MEDIANO	QUEDA	BENEFICIO P5	PF P5	
Entradas perdidas	40,741 \$	95%	37,518 \$	1.52	pasa
Costes hasta el doble	36,834 \$	86%	36,588 \$	1.47	pasa
Ejecuciones degradadas	38,910 \$	91%	38,054 \$	1.51	pasa
Spread más ancho	32,335 \$	75%	32,223 \$	1.40	pasa

Beneficio neto — Entradas perdidas

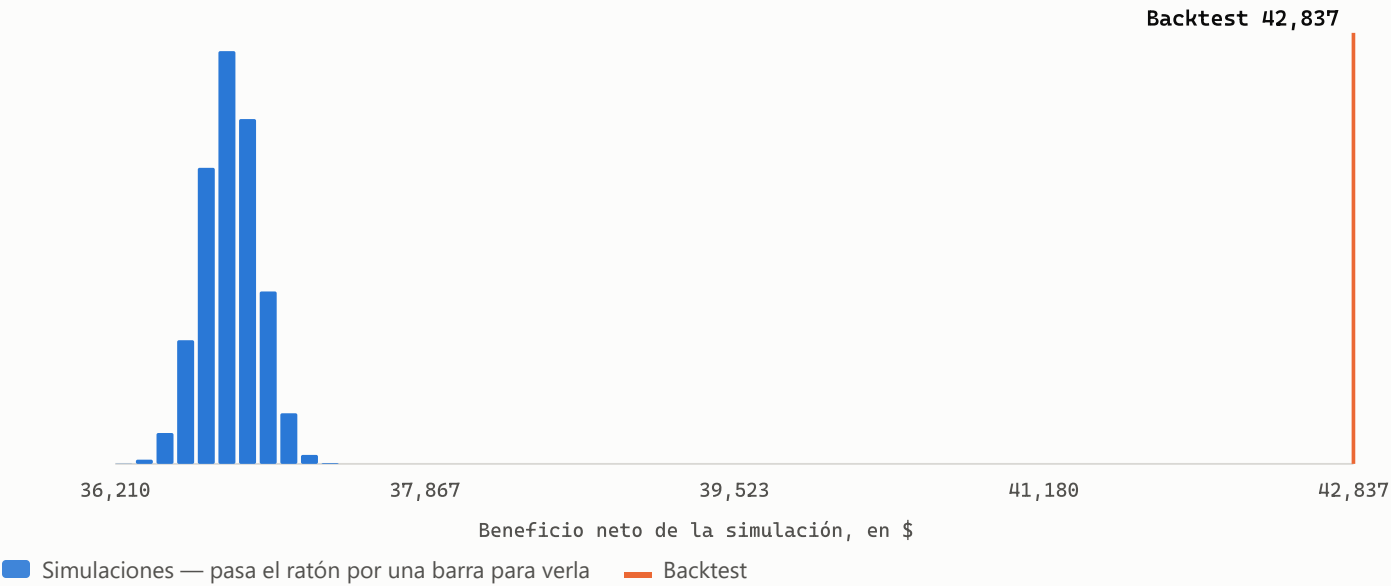


Entradas perdidas — IS vs OOS

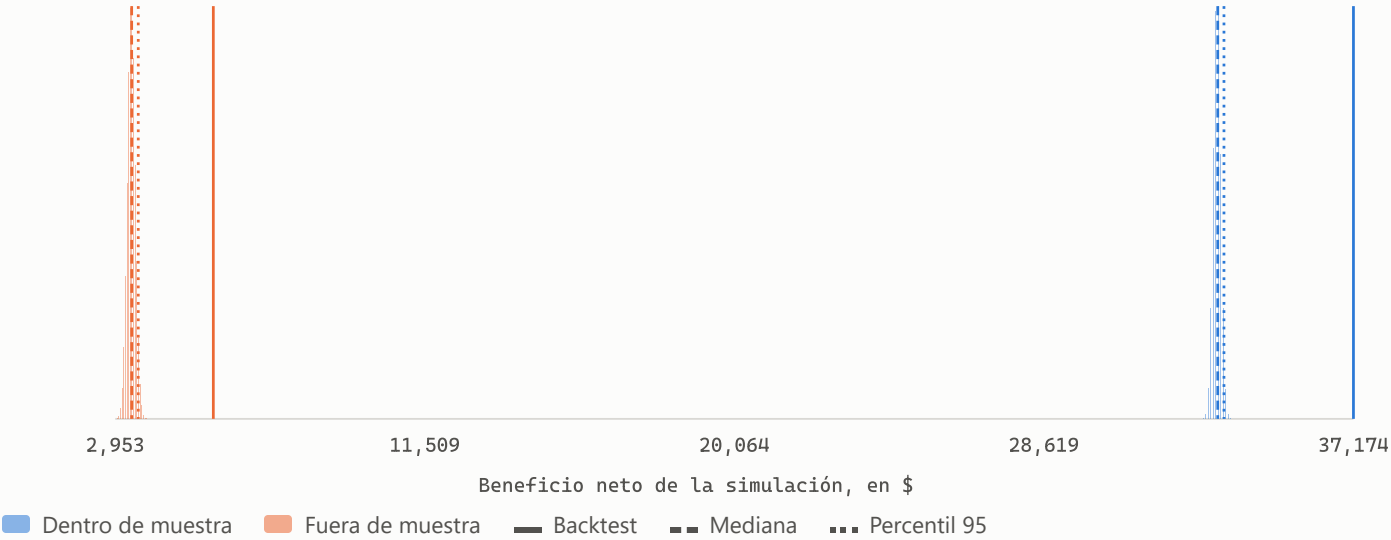


MUESTRA	OPERACIONES	BACKTEST	MEDIANA	PERCENTIL 95
Dentro de muestra	690	3.717e+04	3.539e+04	3.777e+04
Fuera de muestra	292	5,663	5,373	7,091

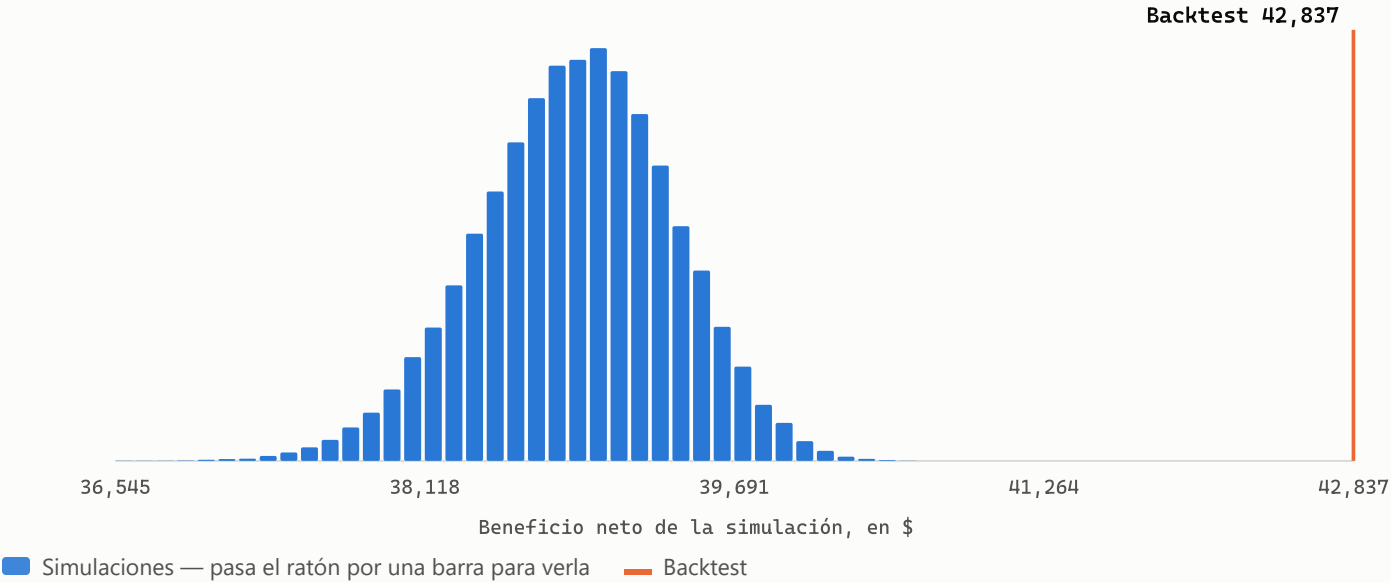
Beneficio neto — Costes hasta el doble



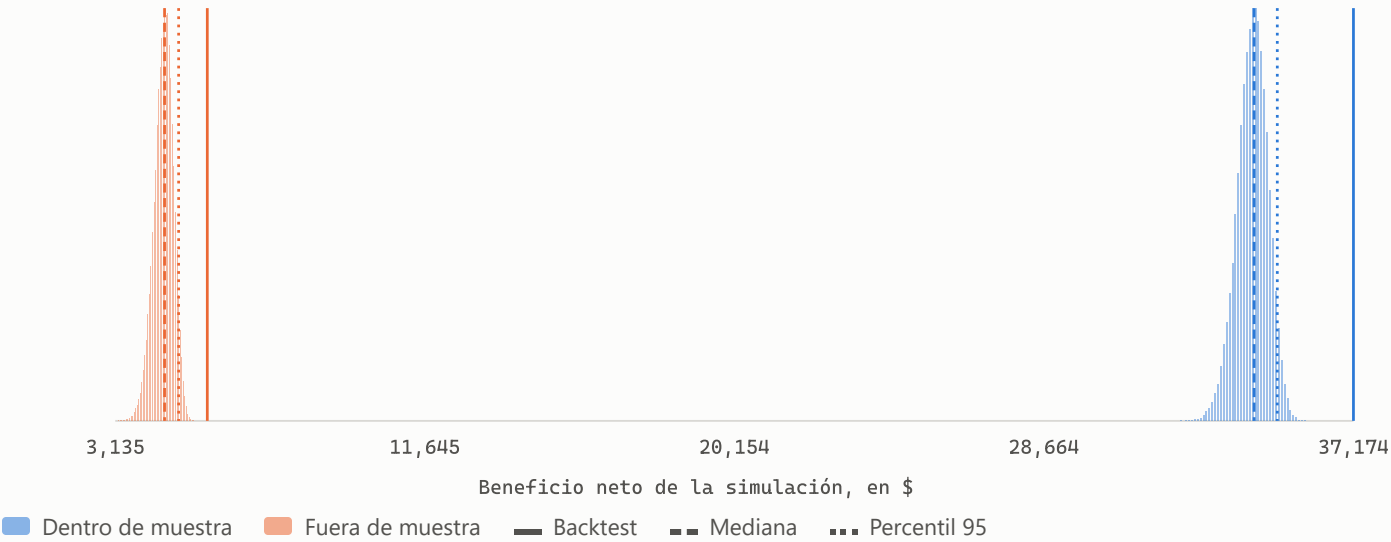
Costes hasta el doble — IS vs OOS



Beneficio neto — Ejecuciones degradadas

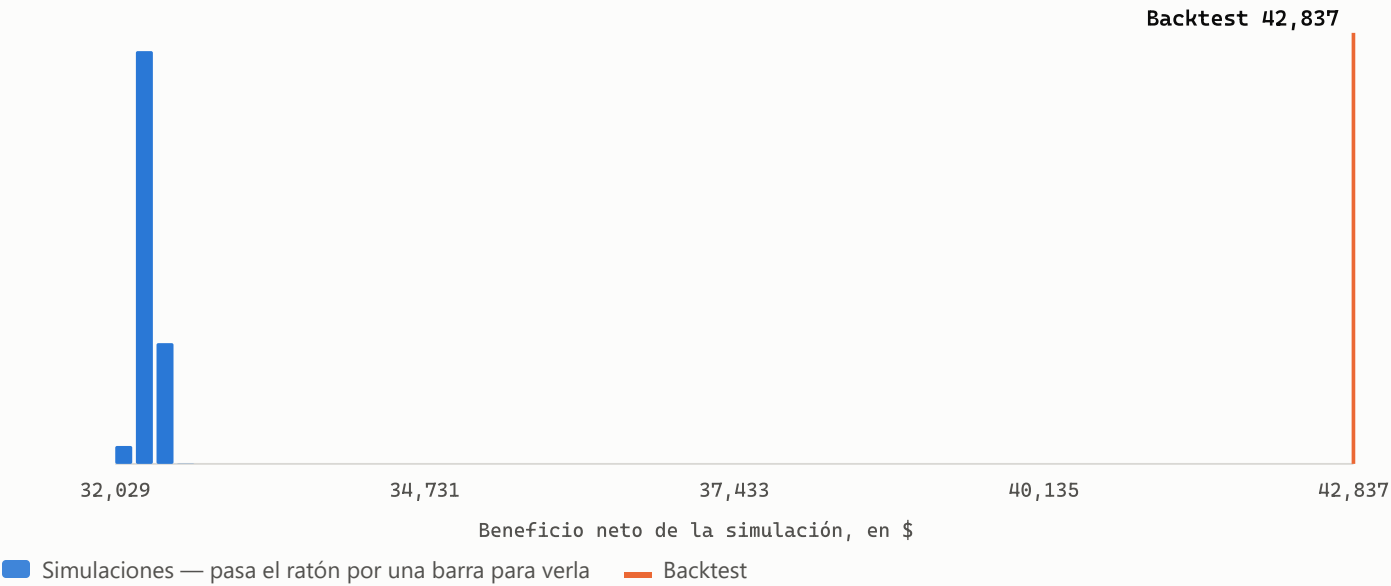


Ejecuciones degradadas — IS vs OOS

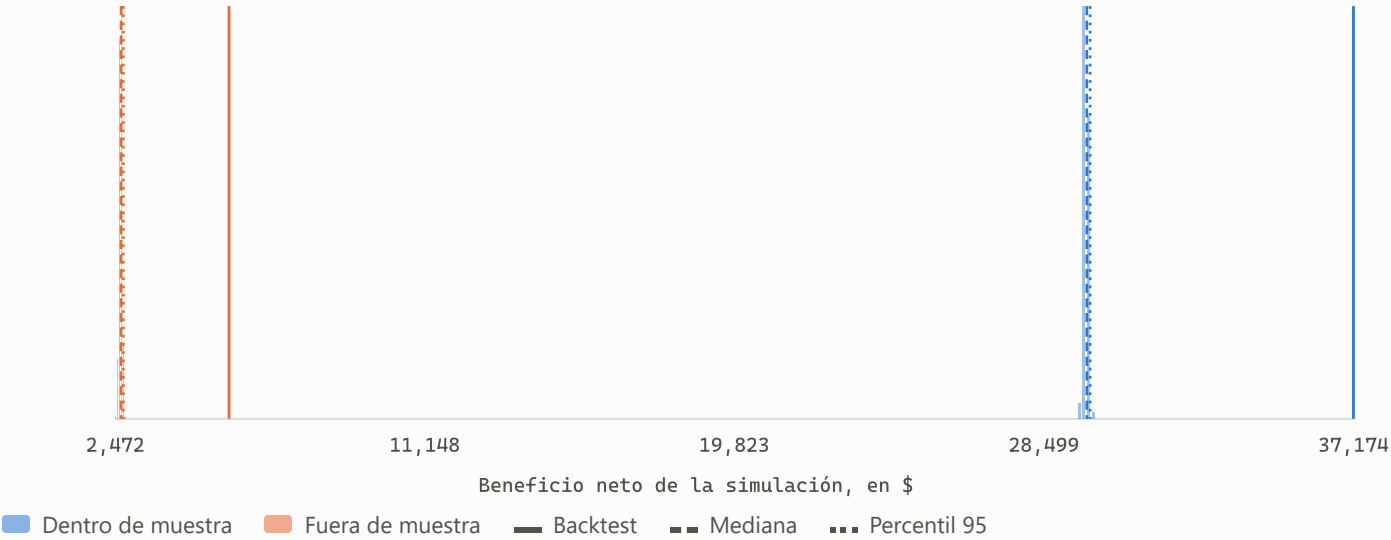


MUESTRA	OPERACIONES	BACKTEST	MEDIANA	PERCENTIL 95
Dentro de muestra	690	3.717e+04	3.444e+04	3.508e+04
Fuera de muestra	292	5,663	4,489	4,878

Beneficio neto — Spread más ancho



Spread más ancho — IS vs OOS



MUESTRA	OPERACIONES	BACKTEST	MEDIANA	PERCENTIL 95
Dentro de muestra	690	3.717e+04	2.97e+04	2.98e+04
Fuera de muestra	292	5,663	2,635	2,695

Familia D — suerte de régimen

0

sin vetos

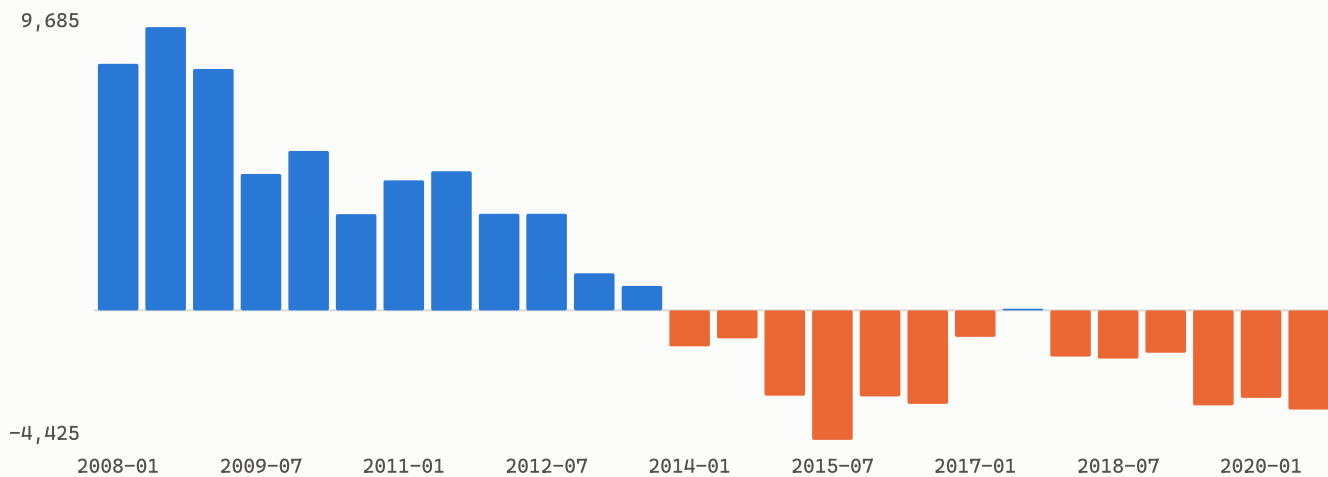
Sub-score familia D de esta familia

- **Aviso · Ventanas móviles** — Sólo el 50% de las ventanas móviles aguanta en positivo en su percentil 5 (se pide 70%).

Las familias A a C dan por hecho que el edge es el mismo siempre. Ésta lo ataca: mira dónde vivió, en el calendario y en la volatilidad del mercado.

Ventanas móviles de 24 meses

percentil 5 del beneficio en cada ventana · 50% en positivo



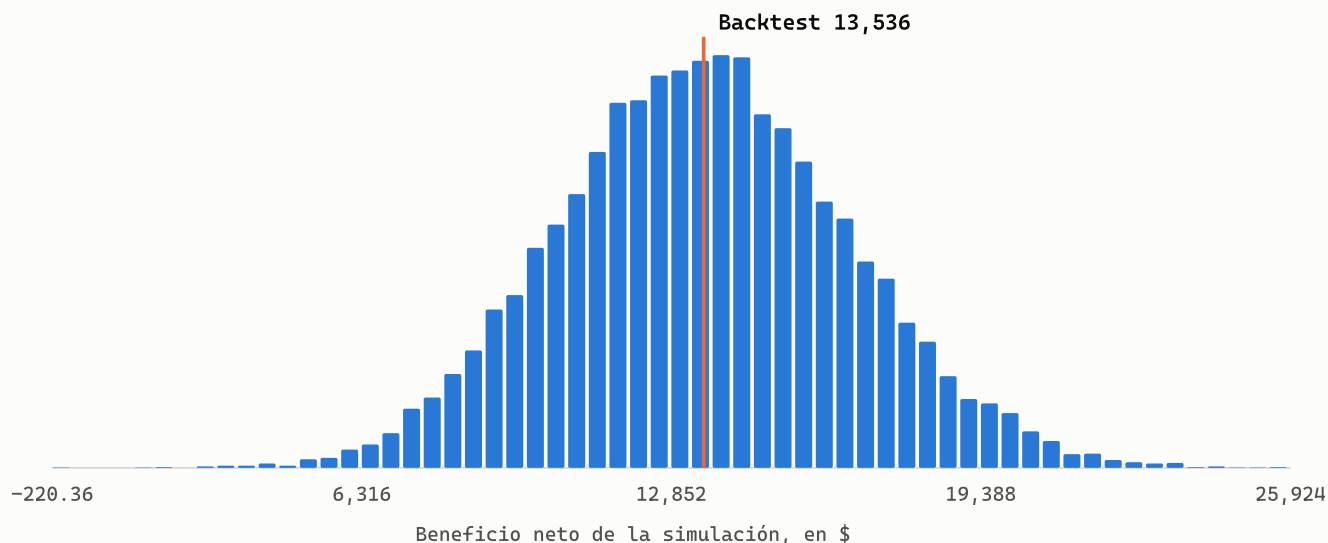
Positivo Negativo

Bloques que no se solapan

Las ventanas solapadas suavizan; estos bloques no comparten ni una operación, así que un periodo malo de verdad aparece aquí y no allí.

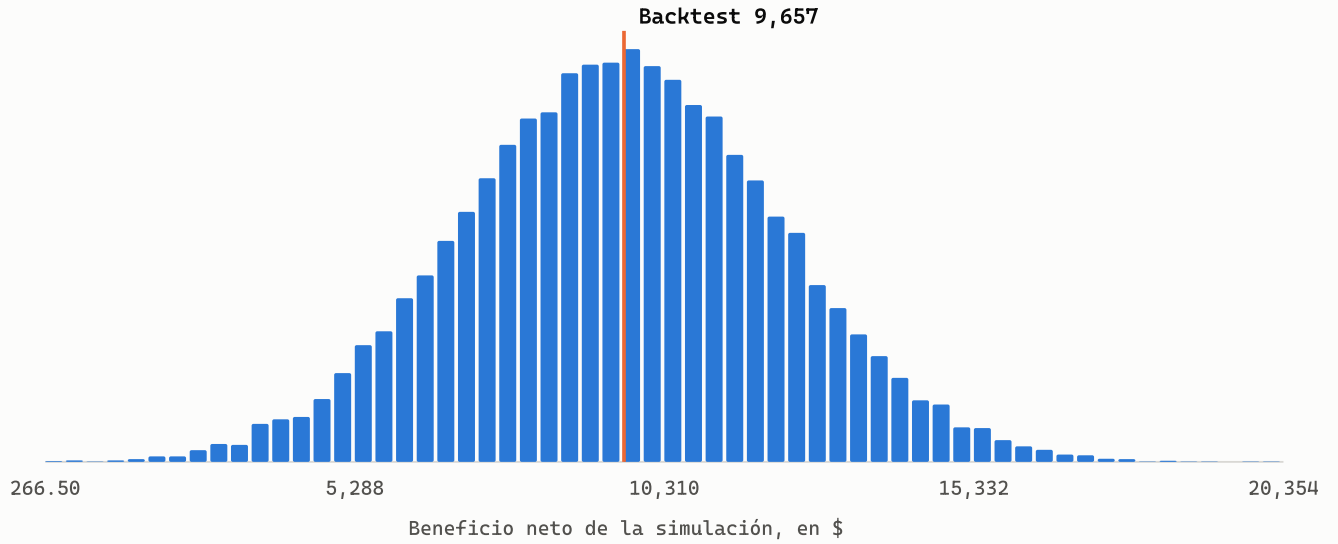
DESDE	OPERACIONES	MEDIANA	PERCENTIL 5	PF P5
2008-01-01	161	13,536 \$	8,594 \$	1.75
2010-01-01	163	9,629 \$	5,430 \$	1.52
2012-01-01	139	8,670 \$	3,308 \$	1.28
2014-01-01	133	3,276 \$	-1,276 \$	0.89
2016-01-01	94	1,842 \$	-2,913 \$	0.71
2018-01-01	105	2,624 \$	-1,579 \$	0.86
2020-01-01	111	1,016 \$	-2,911 \$	0.75

Beneficio remuestreado — bloque desde 2008-01-01



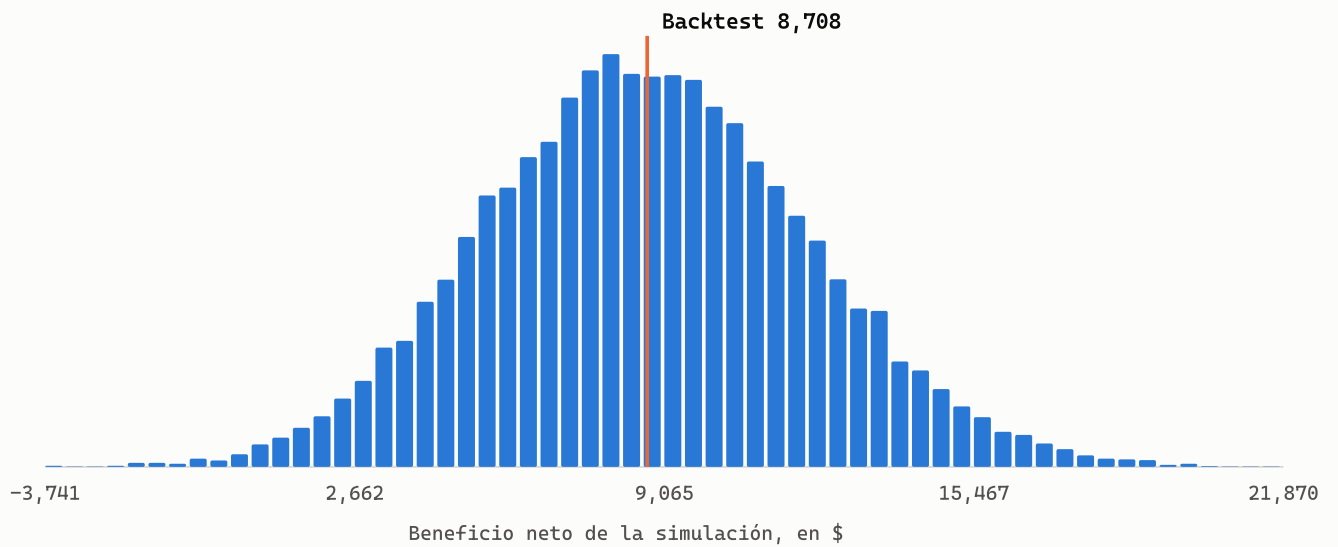
■ Simulaciones — pasa el ratón por una barra para verla — Backtest

Beneficio remuestreado — bloque desde 2010-01-01



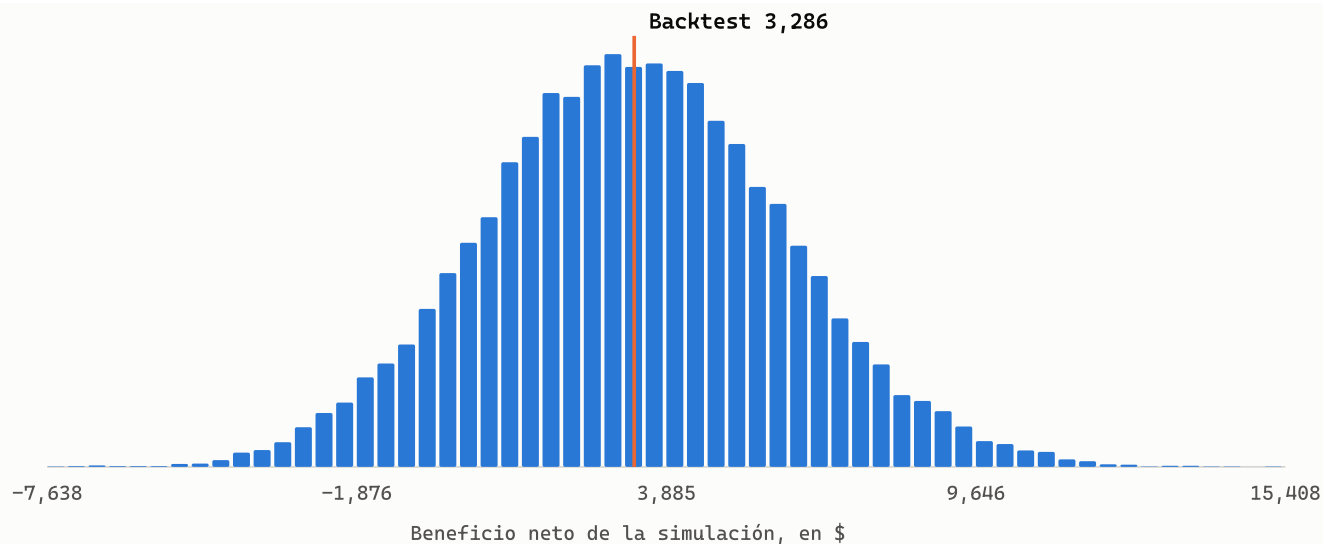
■ Simulaciones — pasa el ratón por una barra para verla — Backtest

Beneficio remuestreado — bloque desde 2012-01-01

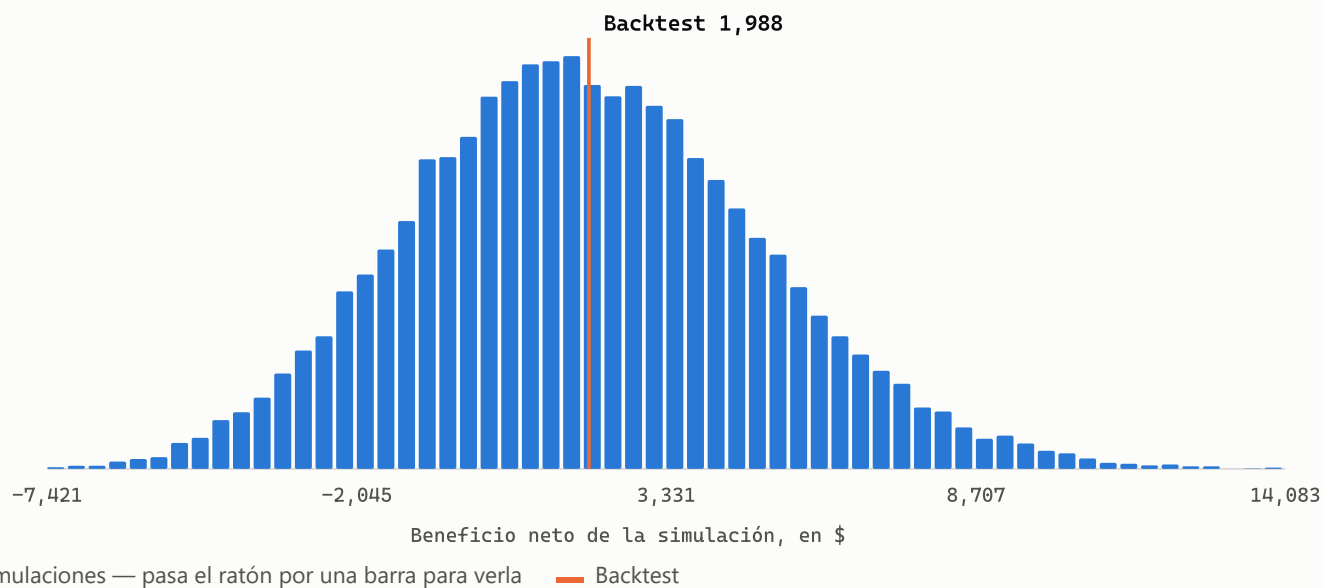


■ Simulaciones — pasa el ratón por una barra para verla — Backtest

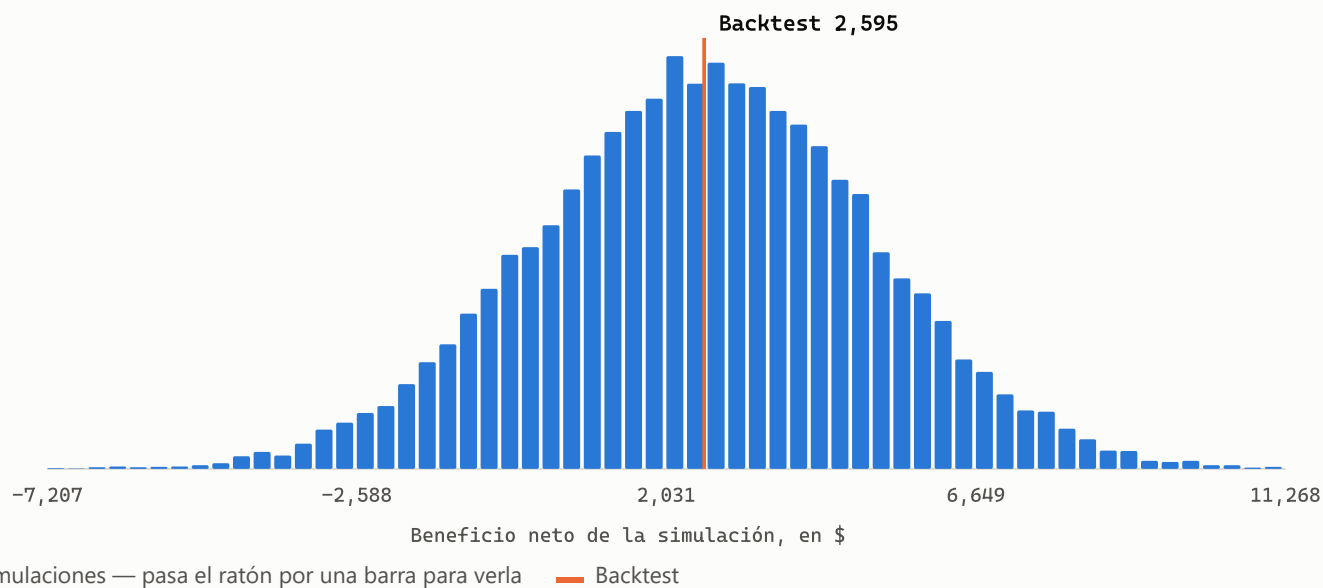
Beneficio remuestreado — bloque desde 2014-01-01



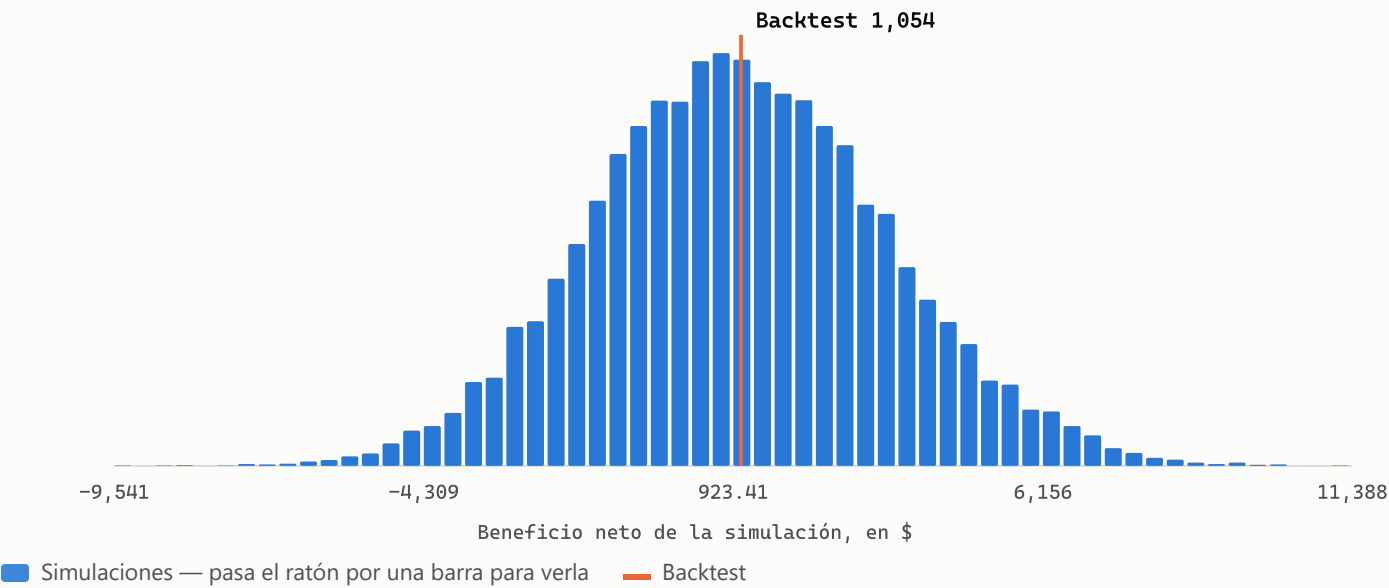
Beneficio remuestreado — bloque desde 2016-01-01



Beneficio remuestreado — bloque desde 2018-01-01



Beneficio remuestreado — bloque desde 2020-01-01



Curva de equity con las ventanas marcadas

cada línea discontinua abre un bloque de la tabla de arriba

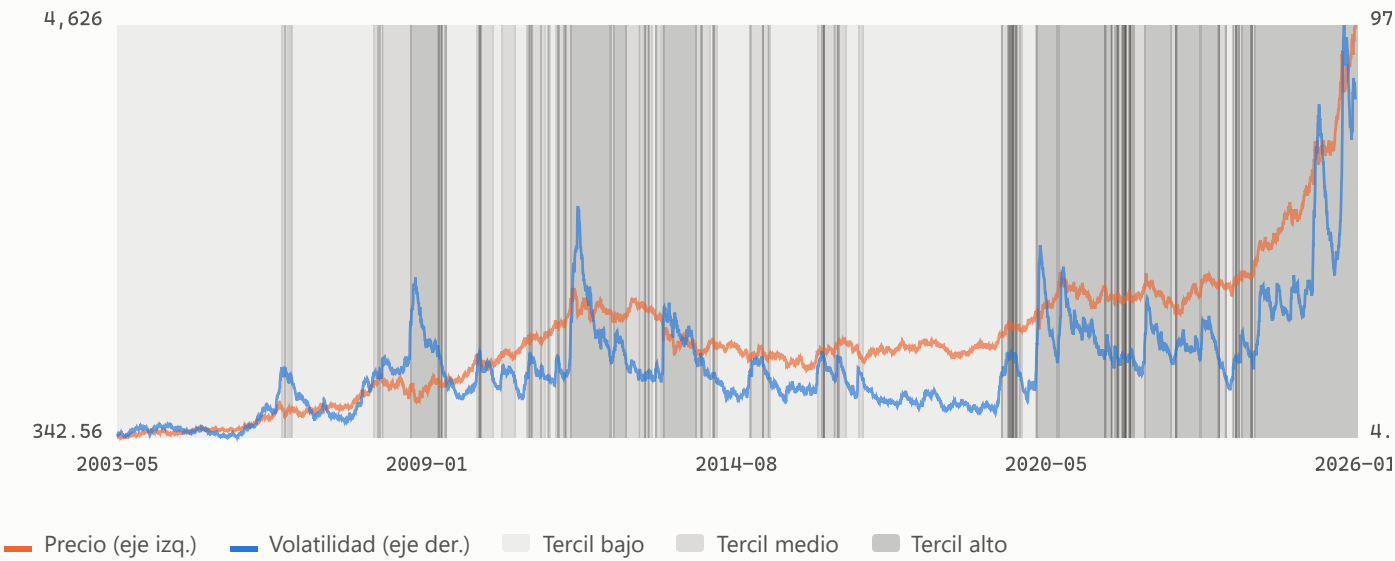


Régimen de volatilidad

ATR(20) diario, en terciles. Cortes en 18.13 y 22.72; cobertura 100.0%.

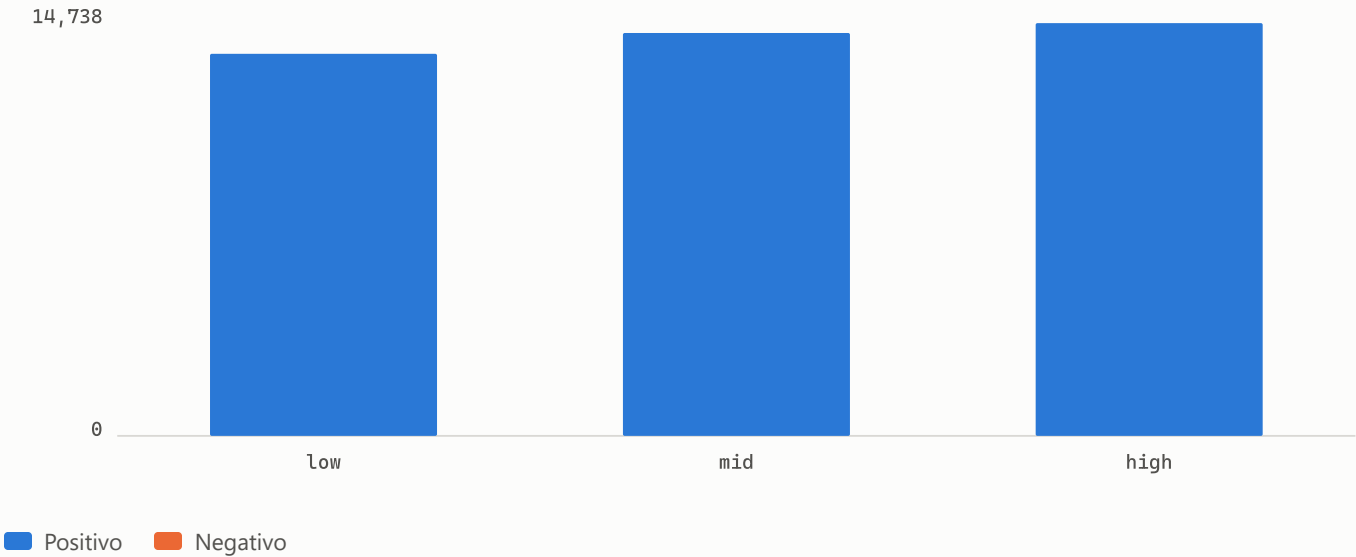
Precio y volatilidad, por tercil

pasa el ratón por una línea para resaltarla



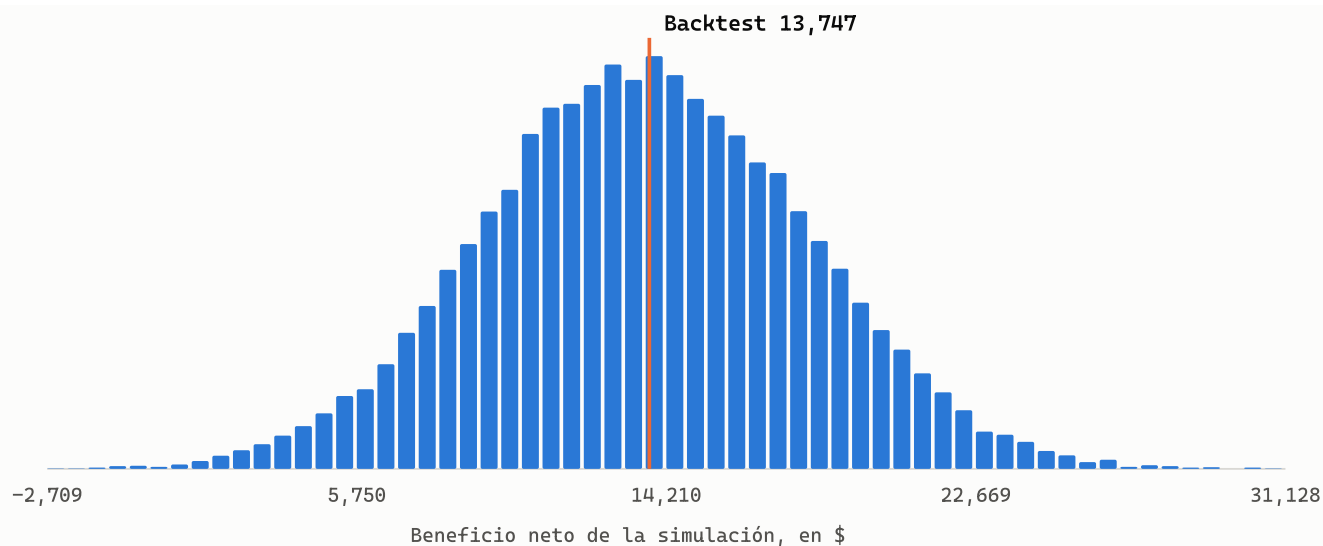
Beneficio mediano remuestreado por tercil de volatilidad

el 34% del beneficio sale de un tercil

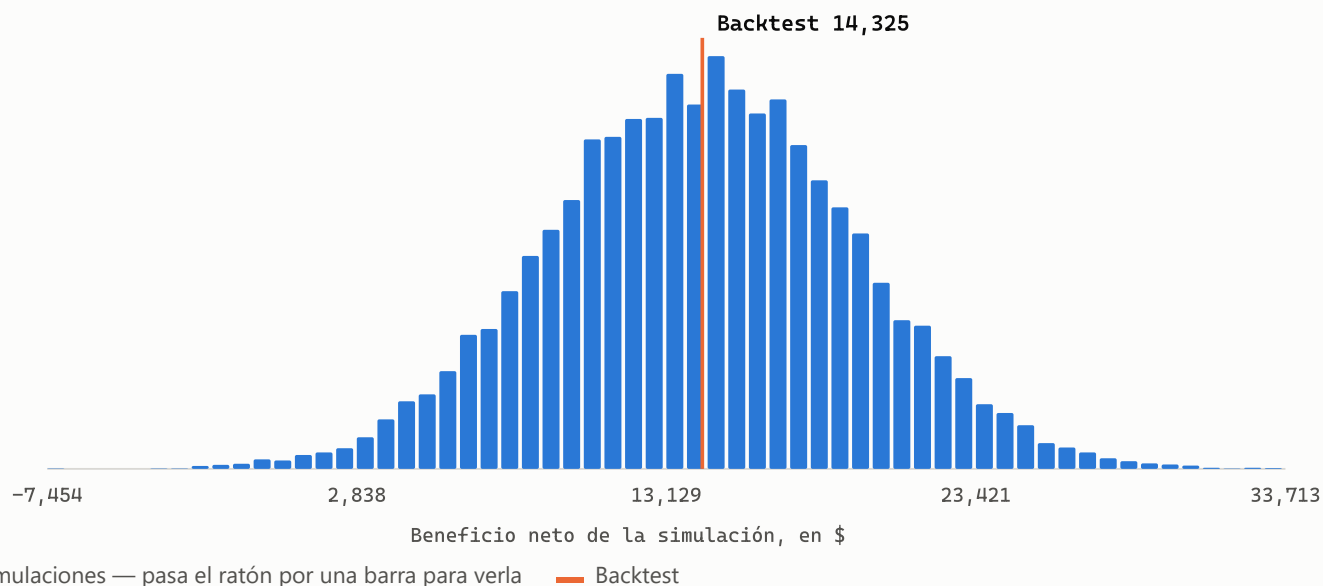


TERCIL	OPERACIONES	BENEFICIO REAL	MEDIANA	PERCENTIL 5	PF P5
low	327	13,747 \$	13,645 \$	6,494 \$	1.24
mid	327	14,325 \$	14,386 \$	6,005 \$	1.22
high	328	14,765 \$	14,738 \$	7,407 \$	1.25

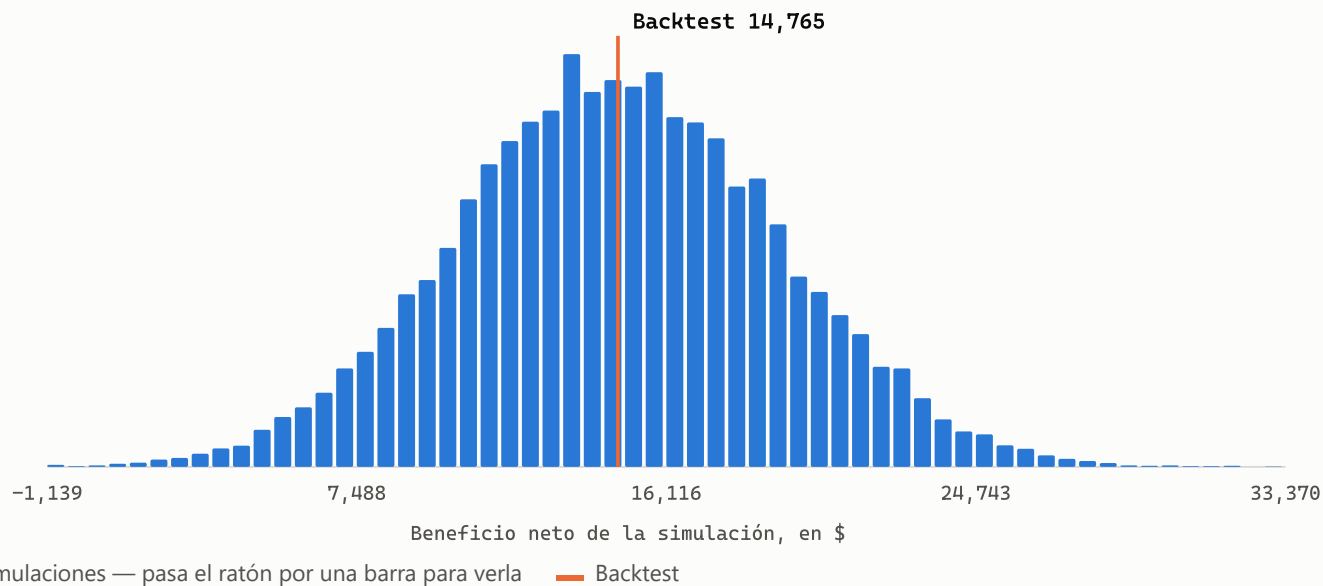
Beneficio remuestreado — tercil low



Beneficio remuestreado — tercil mid



Beneficio remuestreado — tercil high



El peor camino posible

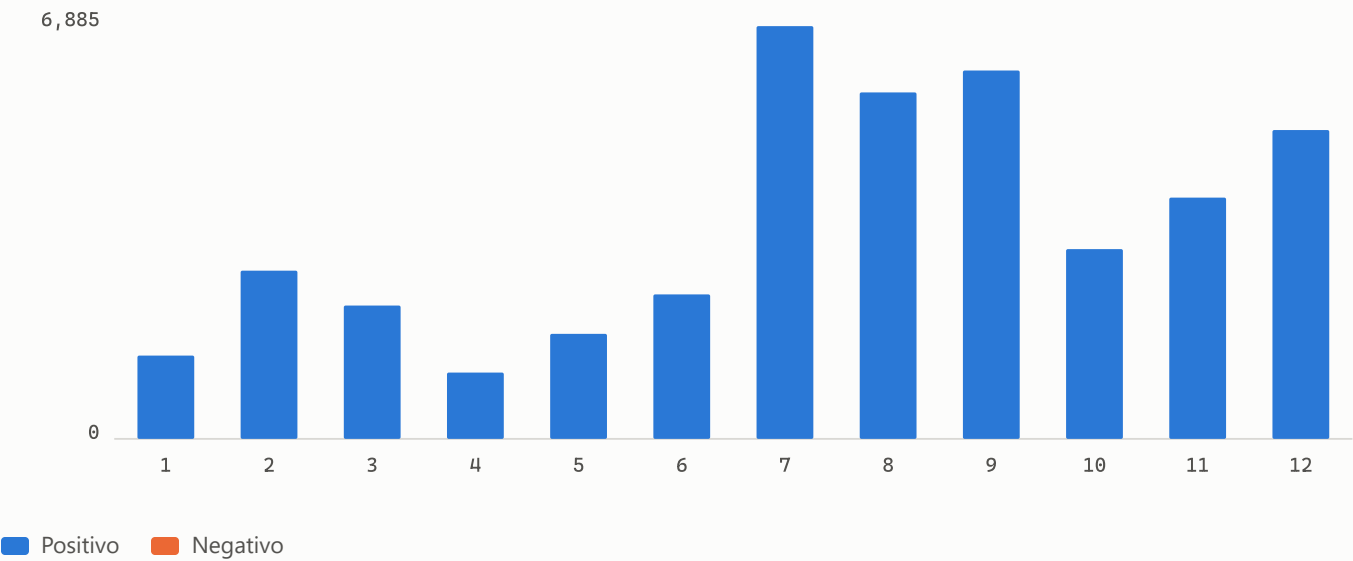
Cosiendo un mal tramo de cada bloque de 24 meses en uno solo. Qué tan malo es "malo" no tiene una respuesta de principio, así que se enseña a varias severidades en vez de elegir una — si el drawdown apenas se mueve entre columnas, la referencia es estable; si se dispara, esa inestabilidad ya es un hallazgo. No es un veto: es una referencia de estrés.

PERCENTIL POR BLOQUE	DRAWDOWN	BENEFICIO NETO	BLOQUES COSIDOS
1%	16.8%	-6,984 \$	7
5%	9.9%	6,677 \$	7
10%	5.6%	13,981 \$	7
25%	2.9%	26,425 \$	7

Para comparar: el drawdown real fue 2.5% y el percentil 95 reordenando (Familia A) es 4.2%.

Beneficio por mes del año

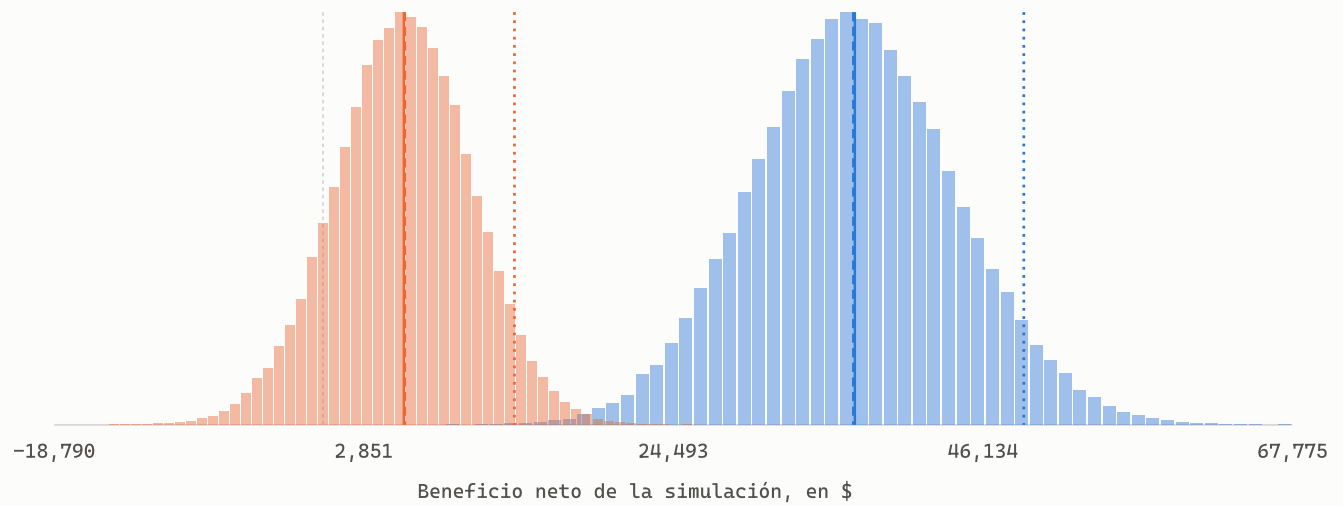
sólo diagnóstico: con doce meses, el mejor de un reparto al azar ya parece notable



Degradación dentro / fuera de muestra

Mismo remuestreo i.i.d. que la Familia B, repetido aquí porque esta familia es la que mira el tiempo: el número no cambia, solo el sitio en el que se lee.

Beneficio remuestreado — IS vs OOS



Dentro de muestra Fuera de muestra Backtest Mediana Percentil 95

MUESTRA	OPERACIONES	BACKTEST	MEDIANA	PERCENTIL 95
Dentro de muestra	690	3.717e+04	3.708e+04	4.9e+04
Fuera de muestra	292	5,663	5,718	1.339e+04

Familia E — significación

100

Sub-score familia E

sin vetos

de esta familia

Ninguna prueba de esta familia falló ni avisó.

La **Probabilistic Sharpe Ratio (PSR)** es la probabilidad de que el Sharpe real de la estrategia sea mayor que un valor de referencia (aquí, cero) — no una probabilidad de ganar dinero, sino de que el edge observado no sea ruido. A diferencia de un contraste clásico, tiene en cuenta cuántas operaciones hay y la forma exacta de su distribución: la asimetría y las colas gordas entran en la cuenta, así que un sistema cuyo beneficio depende de cuatro operaciones enormes saca menos PSR que otro con el mismo Sharpe repartido entre muchas — el número castiga la concentración que el Sharpe por sí solo no ve.

QUÉ	VALOR
PSR	1.0000
Objetivo / veto	0.95 / 0.90
Sharpe por operación	0.1599

Asimetría / curtosis	0.64 / 8.76
Operaciones	982
P(Sharpe>0) remuestreando	1.0000
Diferencia analítica vs remuestreo	0.0000

Las dos últimas filas son la comprobación cruzada: si coinciden, la conclusión no depende de la aproximación normal; si se separan, las operaciones son lo bastante asimétricas como para que sí dependa — y eso es un hallazgo, no un error.

Aquí no hay Deflated Sharpe. Necesitaría saber cuántas estrategias se probaron durante la generación. Ese número no existe en esta fase y fabricarlo daría una cifra creíble y falsa.

De dónde sale cada número. PSR, Sharpe, asimetría y curtosis se calculan directamente sobre las operaciones reales del backtest, en su propio orden — no son el resultado de ninguna simulación, son una fórmula analítica aplicada una vez. `P(Sharpe>0) remuestreando` es la única cifra simulada de esta familia: el percentil de Sharpes positivos en el mismo remuestreo i.i.d. que dibuja el histograma de la Familia B (mismas operaciones, mismo modelo, estadístico distinto). La fila de abajo compara ambas.

Datos y método

QUÉ	VALOR
simulaciones por prueba	100,000
cuenta inicial / riesgo por operación	100,000 \$ / 1,000 \$
operaciones exportadas	..\AlgoData\raw\XAUUSD\Results\2026-09-03\trades
barras diarias	..\AlgoData\bars\XAUUSD_DukasM1_Infinox\M30.csv
coste recuperado del propio backtest	9.72 \$ mediana por operación
coste modelado desde assets/	10.72 \$ (ratio 1.10)
modelo de volatilidad	atr
estabilidad, peor número	stationary.dd_pct.99 ±1.6% en 8 repeticiones
semilla	ninguna: cada ejecución usa entropía nueva

Lo que este informe no dice

No mide sobreajuste. No hay Deflated Sharpe ni CSCV, y no los habrá aquí: harían falta todas las estrategias que se probaron durante la generación, y en esta fase no están. Inventar un número de pruebas para poder calcularlos daría un número creíble y falso. Eso vive en el análisis de generación, no aquí.

No valida el edge. Llega dando por hecho que la estrategia ya lo tiene — pasó el decaimiento y el retest — y sólo mide de qué depende: del orden, de qué operaciones salieron, de la ejecución o del régimen.

El techo de drawdown es provisional. Está en 10% de la cuenta como marcador de posición hasta que las reglas de la prop firm lo fijen. Todas las familias se recalculan cambiando un número del config.

Nada de esto se repite igual dos veces. No hay semilla, a propósito. La sección de estabilidad dice cuánto se mueven los números que deciden; si se mueven demasiado, la respuesta es subir las simulaciones, no fijar una semilla y creerse la primera.